



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES



- **Arquitectura de
Software Funcional**
-

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern furniture. A blue square is in the top right corner.

01

- **Arquitectura de Software y Patrones**

• Qué es una “Arquitectura de Sistema”

• ISO/IEC/IEEE 42010 – Descripción de la arquitectura

- Una arquitectura de sistema establece los conceptos o propiedades fundamentales de un sistema en su entorno incorporados en sus elementos, relaciones y en los principios de su diseño y evolución.

• En resumen

- Una arquitectura del sistema es una descripción abstracta de las entidades de un sistema y las relaciones entre esas entidades.
- Una buena Arquitectura de Sistema presenta las siguientes características
 - Satisface las necesidades de las partes interesadas y ofrece valor
 - Se integra fácilmente con otras entidades (otros sistemas)
 - Evoluciona de manera flexible para cumplir con los requisitos cambiantes a lo largo del tiempo
 - Funciona de forma sencilla y fiable.

• ¿Qué es un Patrón?

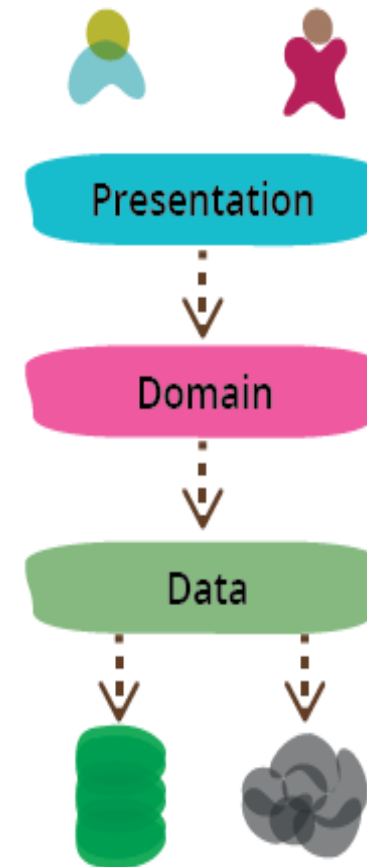
Definición:

- Un **patrón** es una solución probada a un problema recurrente en distintos contextos y tiene una documentación.
- Cuando hay un problema se busca una solución que se ha usado antes
- Reutilizamos aplicaciones que ya han funcionado. Se adapta la solución para un contexto distinto.
- Se documentan las buenas prácticas para usar en otro contexto
- Es un lenguaje en común.
- Nos ayuda a manejar la complejidad.

Ref. Martin Fowler - <https://www.martinfowler.com/architecture/>

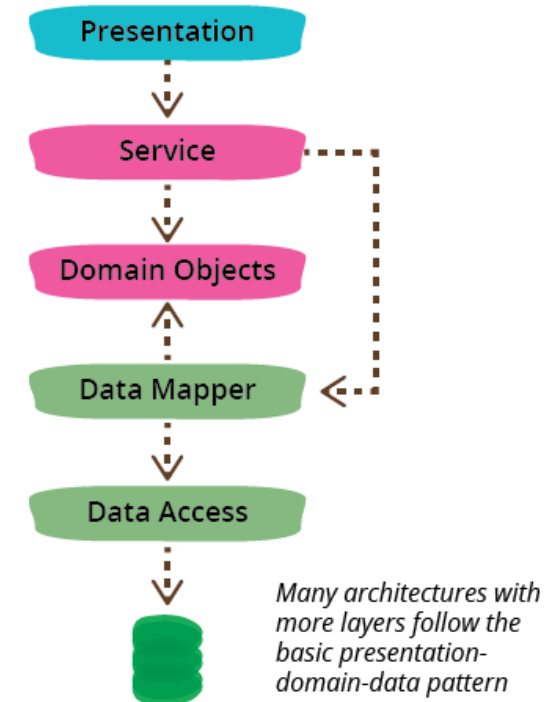
• Patrones de dominio

- Es común ver aplicaciones web divididas en una capa de presentación manejando HTTP que solicita y *render*ea HTML a una capa lógica que contiene validaciones y cálculos, y una capa de acceso que administra los datos persistentes o servicios remotos.
- Se simplifica el problema encapsulando y dividiendo el problema.
- Se simplifica la mantenibilidad.
- Se puede modularizar el problema por capa.
- Mirar nuestra arquitectura como: Presentación-dominio-datos o presentación-dominio y dominio-datos.
- Mirar nuestro software como piezas relativamente independiente.



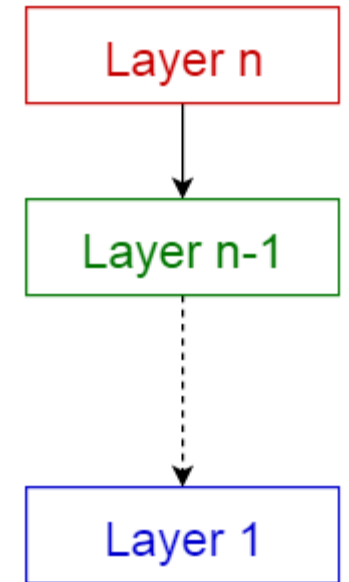
• Patrones de dominio

- Se habla de 3 capas, pero se podrían incluir más.
- Se podría incluir una capa de servicio entre la presentación y dominio.
- O separar la capa de dominio para incluir una capa de mapeo de datos.
- Esencialmente no se pierde el patrón inicial, pero de este dominio, se desprenden distintos patrones.



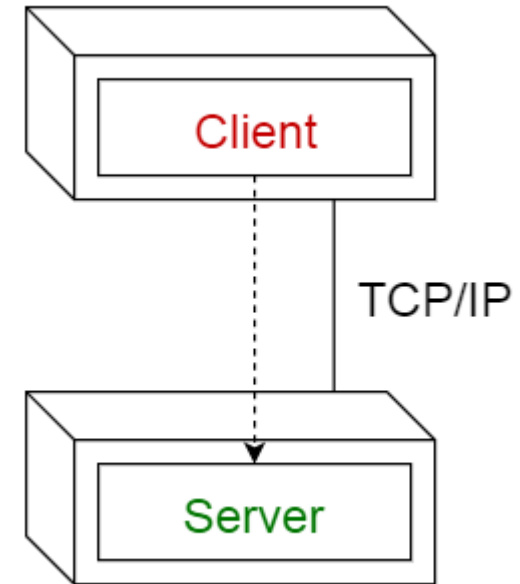
• Patrón de N-Capas

- El patrón arquitectónico Capas ayuda a estructurar las aplicaciones que se pueden descomponer en grupos de subtarefas en la que cada grupo de subtarefas está en un nivel particular de abstracción.



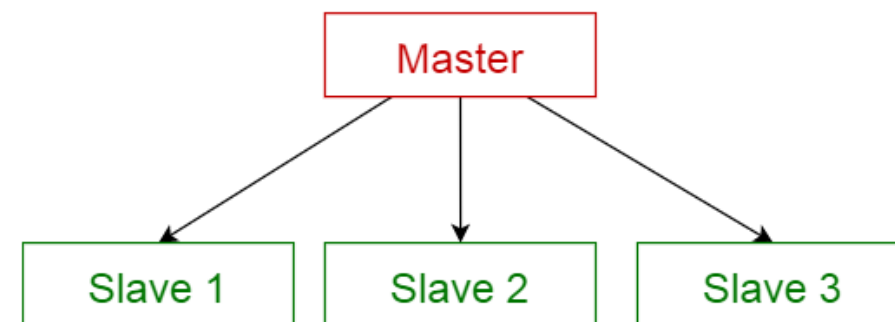
• Patrón Cliente-Servidor

- Este patrón consiste en dos partes; un **servidor** y múltiples **clientes**. El componente del servidor proporcionará servicios a múltiples componentes del cliente. Los clientes solicitan servicios del servidor y el servidor proporciona servicios relevantes a esos clientes. Además, el servidor sigue escuchando las solicitudes de los clientes.
- **Uso:** Aplicaciones en línea como correo electrónico, uso compartido de documentos y banca.



• Patrón Maestro - Esclavo

- Este patrón consiste en dos partes; maestro y esclavos. El componente maestro distribuye el trabajo entre componentes esclavos idénticos y calcula el resultado final de los resultados que devuelven los esclavos.
- **Uso:** En la replicación de la base de datos, la base de datos maestra se considera como la fuente autorizada y las bases de datos esclavas se sincronizan con ella.
- Periféricos conectados a un bus en un sistema informático (unidades maestra y esclava).

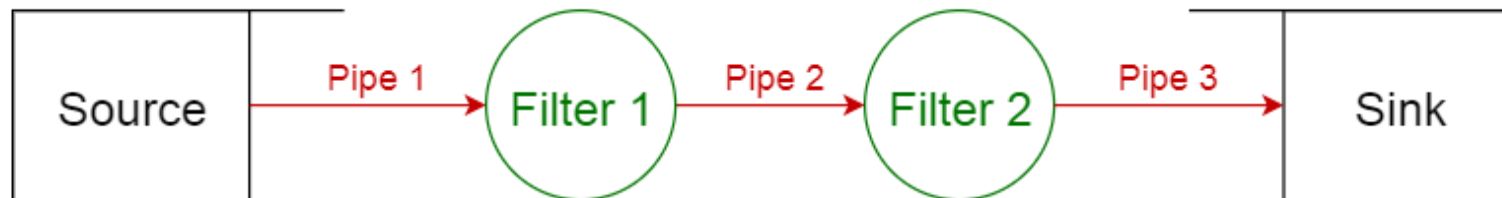


• Patrón Filtro de Tubería

- Este patrón se puede usar para estructurar sistemas que producen y procesan una secuencia de datos. Cada paso de procesamiento se incluye dentro de un componente de filtro . Los datos que se procesarán se pasan a través de las tuberías. Estas tuberías se pueden utilizar para el almacenamiento en búfer o con fines de sincronización.

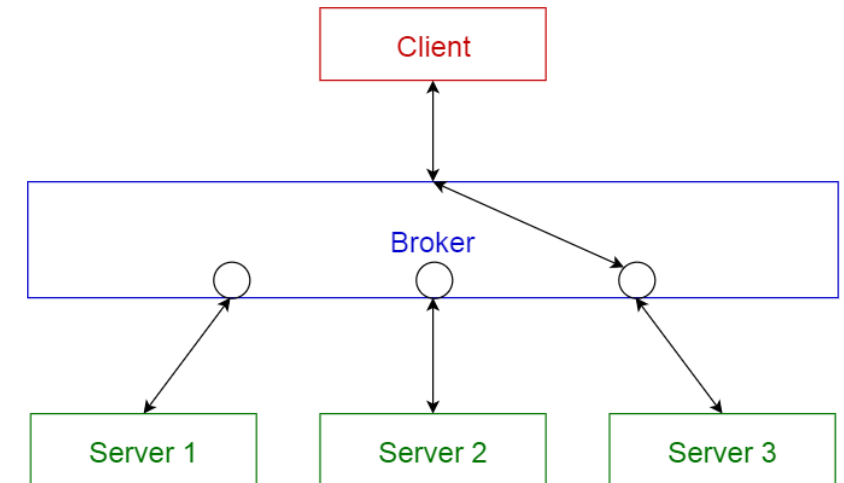
Uso:

- Compiladores Los filtros consecutivos realizan análisis léxico, análisis sintáctico y generación de código.
- Flujos de trabajo en bioinformática.



• Patrón del Agente

- Este patrón se usa para estructurar sistemas distribuidos con componentes desacoplados. Estos componentes pueden interactuar entre sí mediante invocaciones de servicios remotos. Un componente de intermediario es responsable de la coordinación de la comunicación entre los componentes.
- Los servidores publican sus capacidades (servicios y características) a un intermediario. Los clientes solicitan un servicio del intermediario y el intermediario redirecciona al cliente a un servicio adecuado desde su registro.



Uso

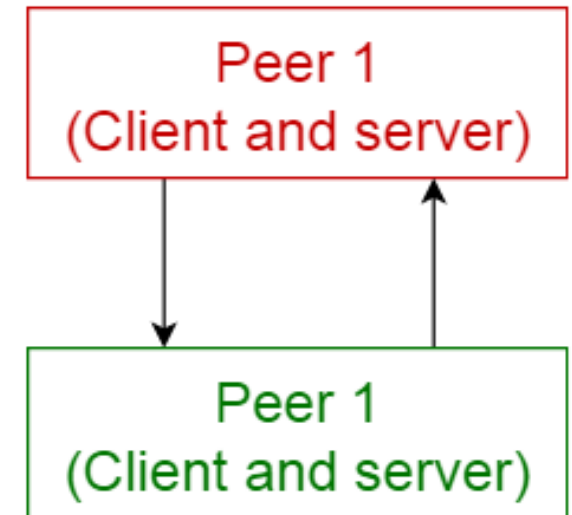
- Software de Message Broker como [Apache ActiveMQ](#), [Apache Kafka](#), [RabbitMQ](#) y [JBoss Messaging](#).

• Patrón Persona a Persona

- En este patrón, los componentes individuales se conocen como pares . Los pares pueden funcionar tanto como un cliente , solicitando servicios de otros pares, y como un servidor , proporcionando servicios a otros pares. Un par puede actuar como un cliente o como un servidor o como ambos, y puede cambiar su rol dinámicamente con el tiempo.

Uso

- Redes de intercambio de archivos como [Gnutella](#) y [G2](#)).
- Protocolos multimedia como [P2PTV](#) y [PDTP](#).

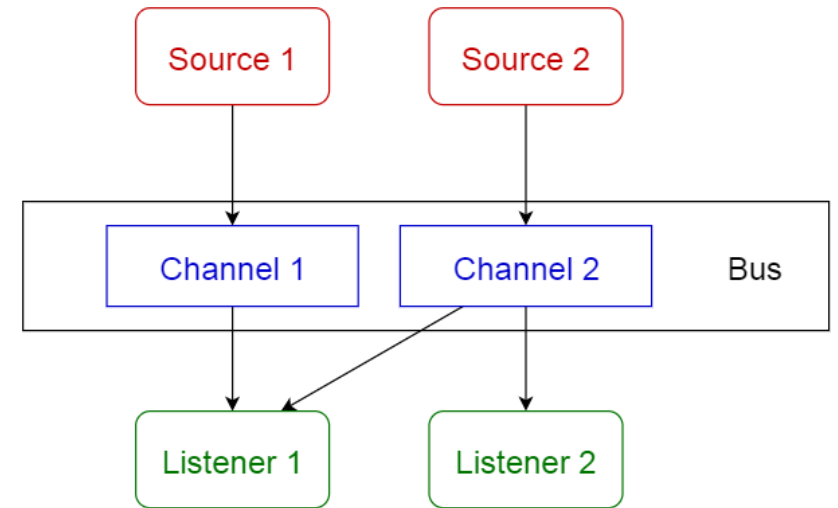


• Patrón de bus de evento

- Este patrón trata principalmente con eventos y tiene 4 componentes principales; fuente de evento , escucha de evento , canal y bus de evento. Las fuentes publican mensajes en canales particulares en un bus de eventos. Los oyentes se suscriben a canales particulares. Los oyentes son notificados de los mensajes que se publican en un canal al que se han suscrito anteriormente.

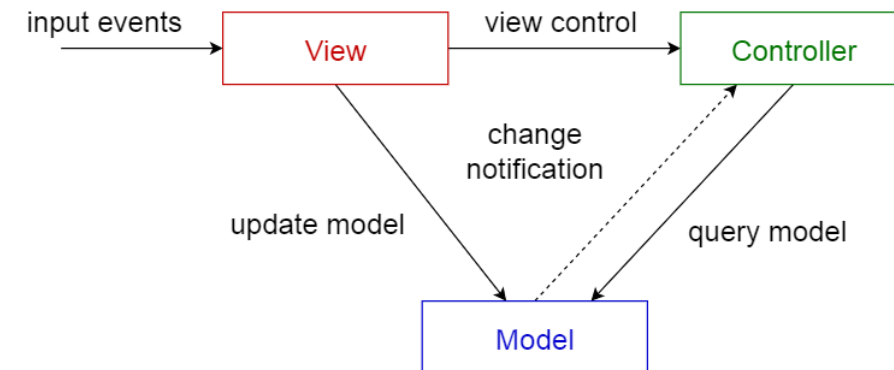
Uso

- Desarrollo de Android
- Servicios de notificación



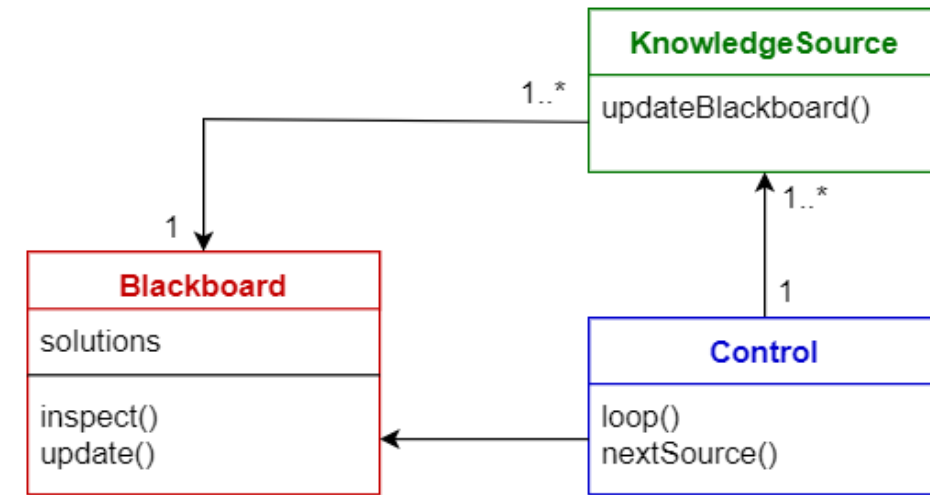
• Patrón Modelo Vista Controlador

- Este patrón, también conocido como patrón MVC, divide una aplicación interactiva en 3 partes, como
 - **modelo** — contiene la funcionalidad y los datos básicos
 - **vista** : muestra la información al usuario (se puede definir más de una vista)
 - **controlador** : maneja la entrada del usuario
- Esto se hace para separar las representaciones internas de información de las formas en que se presenta y acepta la información del usuario. Desacopla los componentes y permite la reutilización eficiente del código.
- **Uso:** Arquitectura para aplicaciones World Wide Web en los principales lenguajes de programación.
- Marcos web como [Django](#) y [Rails](#)



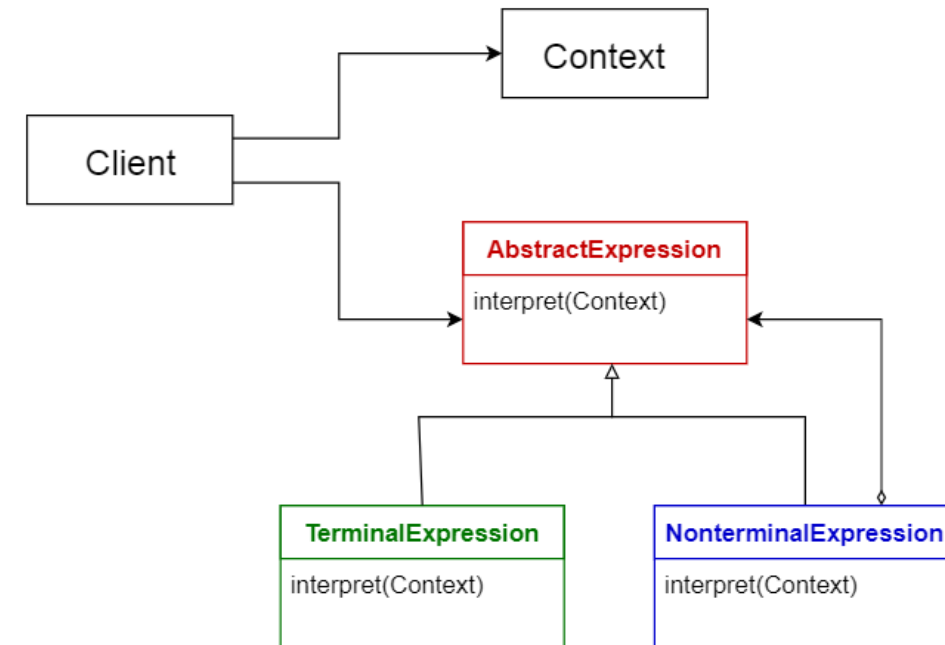
• Patrón Pizarra

- Este patrón es útil para problemas para los que no se conocen estrategias de solución deterministas. El patrón de pizarra consta de 3 componentes principales.
- **pizarra**: una memoria global estructurada que contiene objetos del espacio de solución.
- **fuentes de conocimiento**: módulos especializados con su propia representación
- **componente de control**: selecciona, configura y ejecuta módulos.
- **Uso**: Reconocimiento de voz, Identificación y seguimiento del vehículo, Identificación de la estructura proteica, Sonar señala la interpretación



• Patrón Intérprete

- Este patrón se usa para diseñar un componente que interpreta programas escritos en un lenguaje dedicado. Especifica principalmente cómo evaluar las líneas de programas, conocidas como oraciones o expresiones escritas en un idioma particular. La idea básica es tener una clase para cada símbolo del idioma.
- **Uso**
- Lenguajes de consulta de base de datos como SQL.
- Idiomas utilizados para describir los protocolos de comunicación.



• Comparación de Patrones

Nombre	Ventajas	Desventajas
Capas	Una capa inferior puede ser utilizada por diferentes capas superiores. Las capas facilitan la estandarización ya que podemos definir niveles claramente. Los cambios se pueden hacer dentro de la capa sin afectar a otras capas.	No es universalmente aplicable. En ciertas situaciones, algunas capas pueden tener que ser omitidas.
Cliente-servidor	Bueno para modelar un conjunto de servicios donde los clientes pueden solicitarlos.	Las solicitudes generalmente se manejan en hilos separados en el servidor. La comunicación entre procesos causa sobrecarga ya que diferentes clientes tienen diferentes representaciones.
Maestro-esclavo	Precisión: La ejecución de un servicio se delega a diferentes esclavos, con diferentes implementaciones.	Los esclavos están aislados: no hay estado compartido. La latencia en la comunicación maestro-esclavo puede ser un problema, por ejemplo en sistemas en tiempo real. Este patrón solo se puede aplicar a un problema que puede descomponerse.
Filtro-tubería	Muestra procesamiento concurrente. Cuando la entrada y la salida consisten en flujos, y los filtros comienzan a computar cuando reciben datos. Fácil de agregar filtros. El sistema se puede extender fácilmente. Los filtros son reutilizables. Se pueden construir diferentes tuberías recombinando un conjunto dado de filtros.	La eficiencia está limitada por el proceso de filtro más lento. Sobrecarga de transformación de datos al moverse de un filtro a otro.
Corredor	Permite el cambio dinámico, la adición, eliminación y reubicación de objetos, y hace que la distribución sea transparente para el desarrollador.	Requiere estandarización de descripciones de servicio.
Persona a persona	Soporta computación descentralizada. Muy robusto en caso de falla de un nodo dado. Altamente escalable en términos de recursos y poder de cómputo.	No hay garantía sobre la calidad del servicio, ya que los nodos cooperan voluntariamente. Es difícil garantizar la seguridad. El rendimiento depende del número de nodos.
Bus de eventos	Nuevos editores, suscriptores y conexiones se pueden agregar fácilmente. Efectivo para aplicaciones altamente distribuidas.	La escalabilidad puede ser un problema, ya que todos los mensajes viajan a través del mismo bus de eventos.
Modelo-vista-controlador	Facilita tener múltiples vistas del mismo modelo, que pueden conectarse y desconectarse en tiempo de ejecución.	Aumenta la complejidad. Puede llevar a muchas actualizaciones innecesarias para las acciones del usuario.
Pizarra	Fácil de agregar nuevas aplicaciones. Extender la estructura del espacio de datos es fácil.	Modificar la estructura del espacio de datos es difícil, ya que todas las aplicaciones se ven afectadas. Puede necesitar sincronización y control de acceso.
Intérprete	Es posible un comportamiento altamente dinámico. Bueno para la programabilidad del usuario final. Mejora la flexibilidad, porque reemplazar un programa interpretado es fácil.	Dado que un lenguaje interpretado generalmente es más lento que uno compilado, el rendimiento puede ser un problema.

02

- **Arquitectura de Software Basada en Dominio**

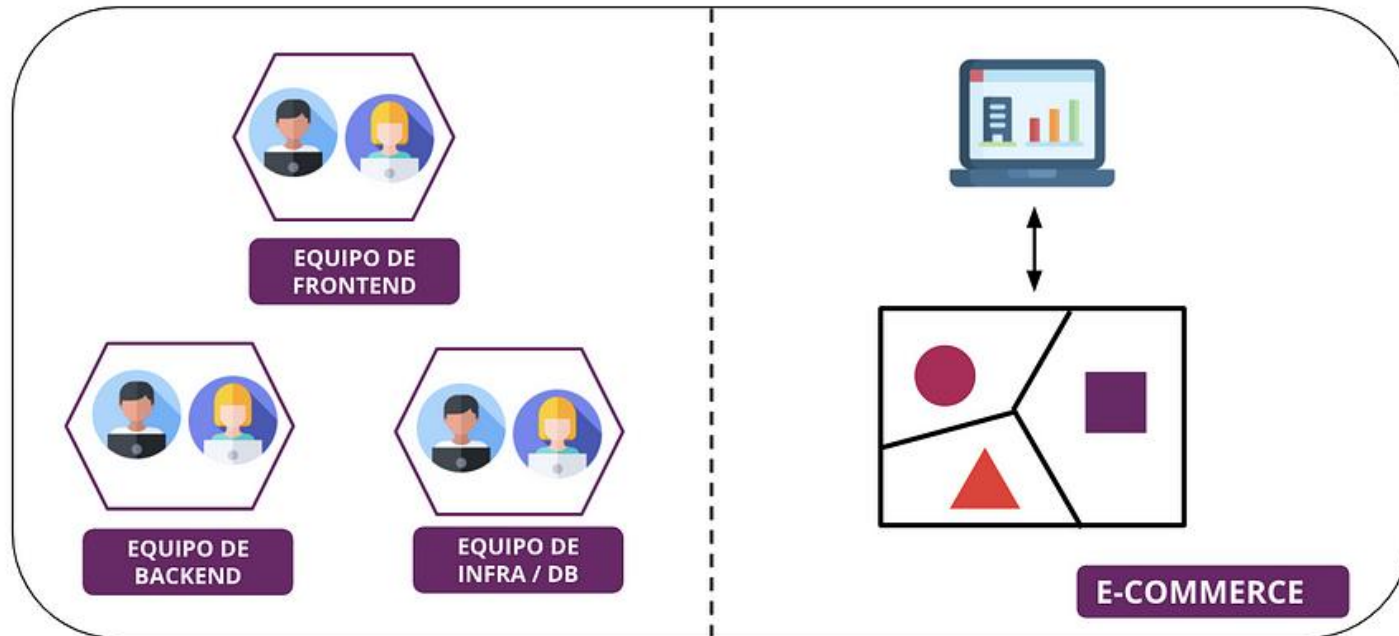


03

- **Modelando la arquitectura del software**

• Ley de Conway

Por ejemplo, un sistema que está dividido en equipos de *frontend*, *backend* y administradores de base de datos (DBA) donde existen silos y problemas de comunicación, van a producir sistemas aislados y fuertemente acoplados al contexto del equipo.



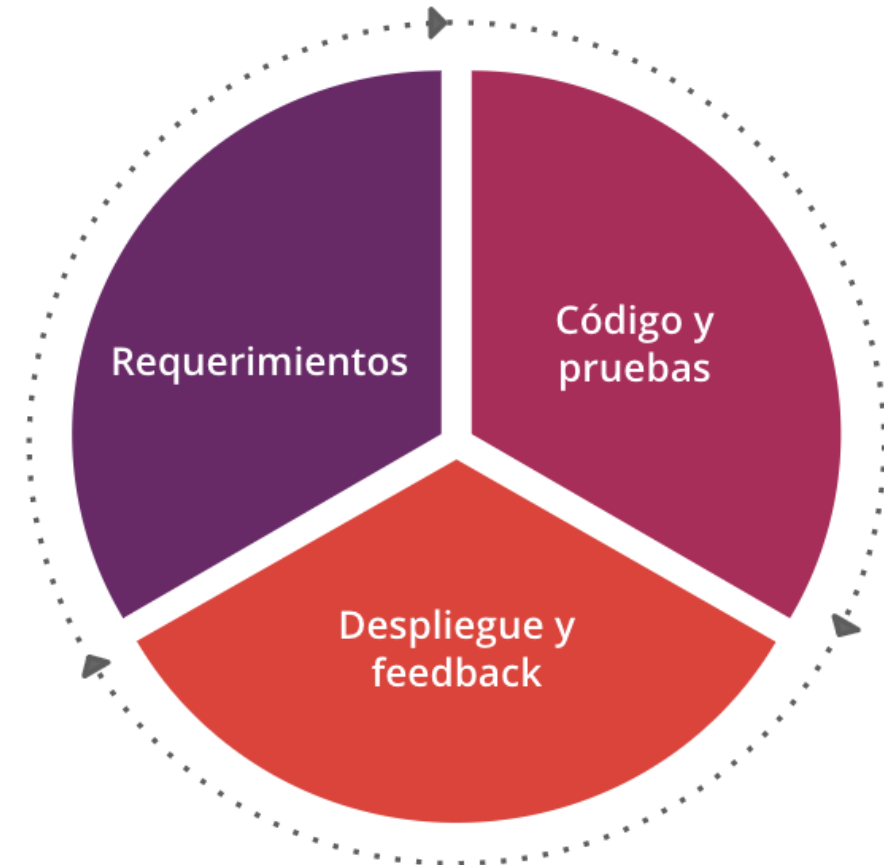
- Es por esto, que debemos armar el negocio y los equipos de tal forma que represente lo que se necesita, por ejemplo, migrar de equipos de proyecto a **equipos de producto**.

- **Arquitectura Ágil**

- **Lo único constante es el cambio.**

(Heráclito de Éfeso, 535 a.C. – 475 a.C.)

- **El software por su naturaleza es cambiante**, cuando iniciamos en un proyecto ágil algunas veces no vemos o no nos preocupamos por conceptos de arquitectura que siempre están presentes, por lo cual el producto se vuelve difícil de cambiar en el futuro.

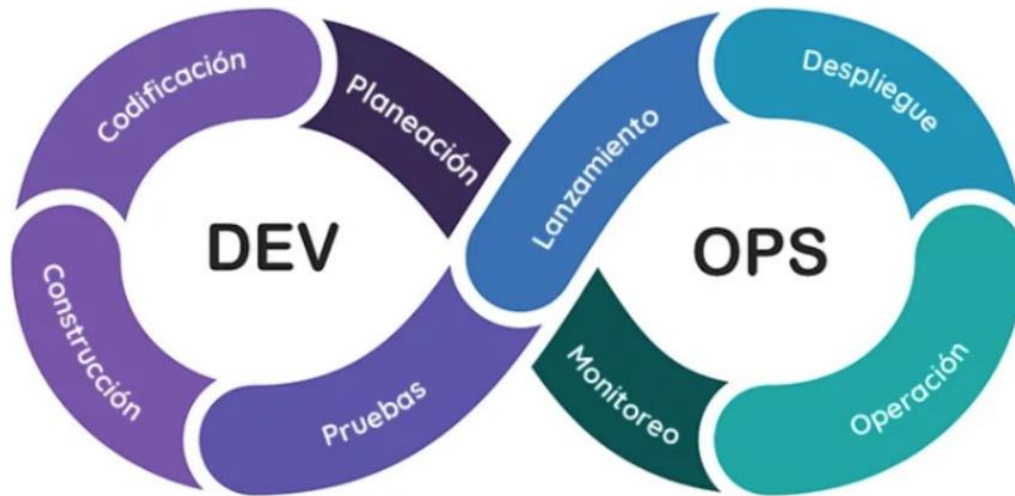


Ciclo de desarrollo en un equipo ágil.

https://miro.medium.com/v2/resize:fit:720/format:webp/1*hGertxkAgHfdnx7zGWOYvw.png

• Arquitectura Ágil

- **La arquitectura no está escrita en piedra** y no es algo rígido, más bien es algo que va evolucionando a medida que el producto se desarrolla y no es trivial identificar dónde debería ir cierta lógica de negocios sin una arquitectura bien definida.



<https://sesitdigital.com/wp-content/uploads/2019/11/Imagen-destacada-SES-39.png>

- Los ingenieros de **DevOps** mejoran la colaboración entre las operaciones y los equipos de desarrollo. Utilizan tecnología, especialmente herramientas de automatización que pueden **aprovechar una infraestructura cada vez más programable y dinámica desde una perspectiva de ciclo de vida**

<https://sesitdigital.com/arquitectura-de-software-y-cultura-devops/>

• Tipos de Arquitectura

- Existen varios tipos de arquitectura, las cuales buscan resolver distintos problemas, y las cuales varían en complejidad, entre estos tipos no existe el concepto de mejor arquitectura, ya que todo “**depende**” **del contexto y del problema** que queremos resolver. Existen distintos tipos de arquitectura, donde podemos destacar a grandes rasgos los **monolitos** y las arquitecturas **distribuidas**.

• Tipos de Arquitectura

Monolítica

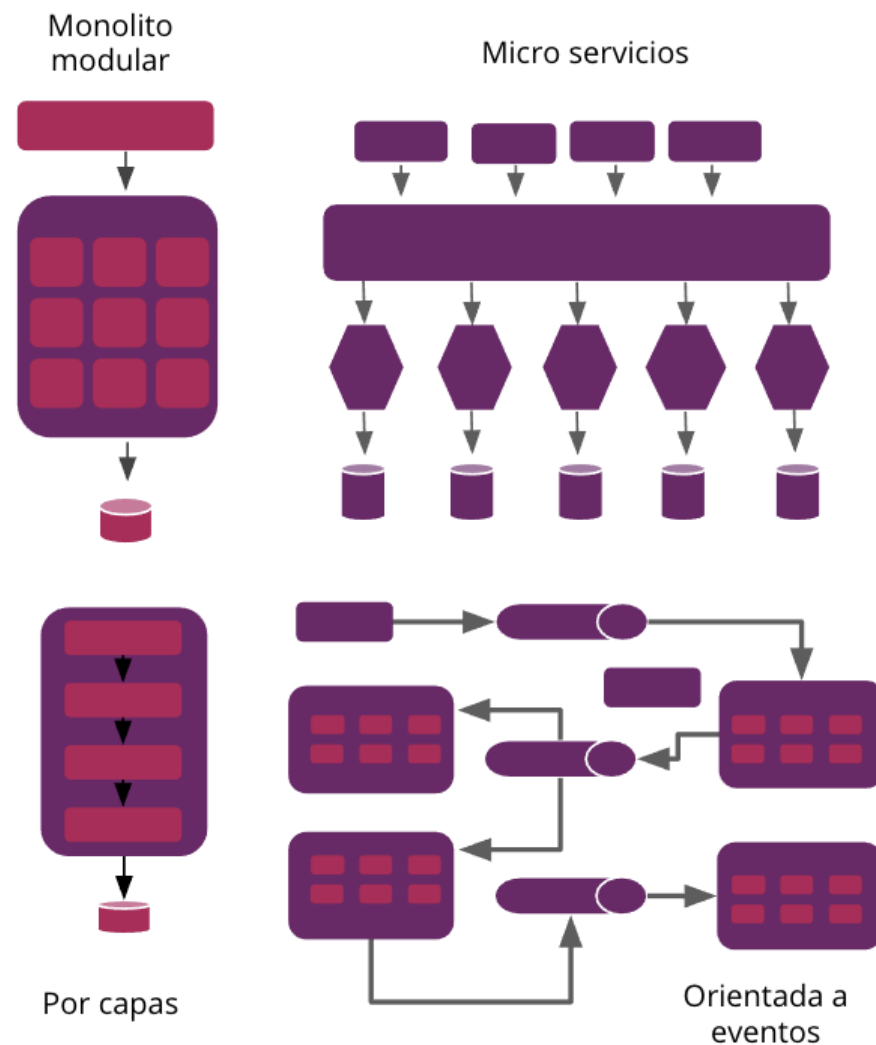
Los monolitos son aquellos que combinan la capa de interfaz del usuario, la capa de negocios y la base de datos en un solo sistema, es decir, es responsable por todos los pasos necesarios para una función en particular. En los monolitos podemos encontrar:

- Arquitectura por capas
- Arquitecturas modulares
- Micro Kernels

Distribuida

Las arquitecturas distribuidas son aquellas que al contrario del monolito sus procesos son distribuidos, es decir si realizamos una consulta en la base de datos, vamos a dividir todos los procesos en distintos nodos, lo que mejorará el performance final de la aplicación. En esta arquitectura podemos encontrar:

- Microservicios
- Arquitectura orientada a servicios
- Arquitectura orientada a eventos



• Diseño Guiado por Dominio



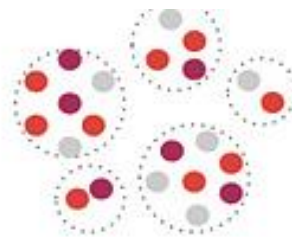
1. Alineamiento

Al modelo de negocios y las necesidades de los usuarios



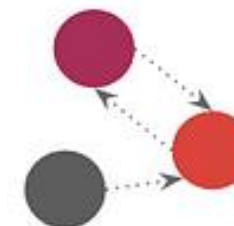
2. Descubrir

La visualización del dominio y la colaboración



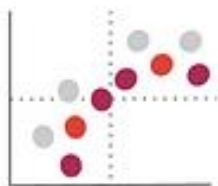
3. Descomponer

El dominio en subdominios



4. Conectar

los subdominios para formar una arquitectura débilmente acoplada



5. Estrategizar

Dominios centrales que diferencian el negocio



6. Organizar

Equipos alrededor de los bounded contexts



7. Definir

Responsabilidades y roles para cada bounded context



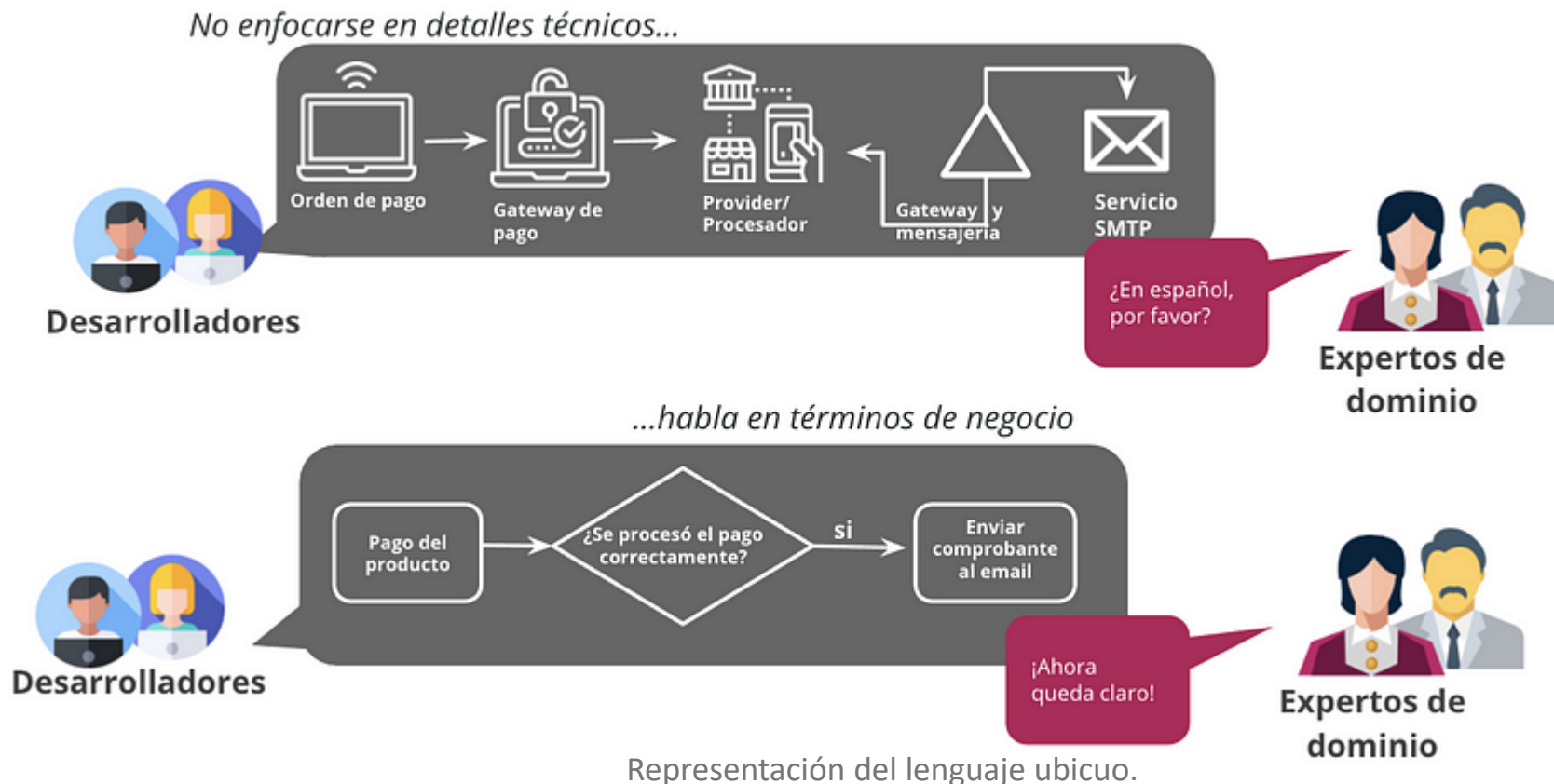
8. Codear

Tus bounded contexts con patrones tácticos

Adaptación del diseño guiado por el dominio. Fuente: <https://github.com/ddd-crew/ddd-starter-modelling-process>

• Lenguaje de negocio vs tecnología

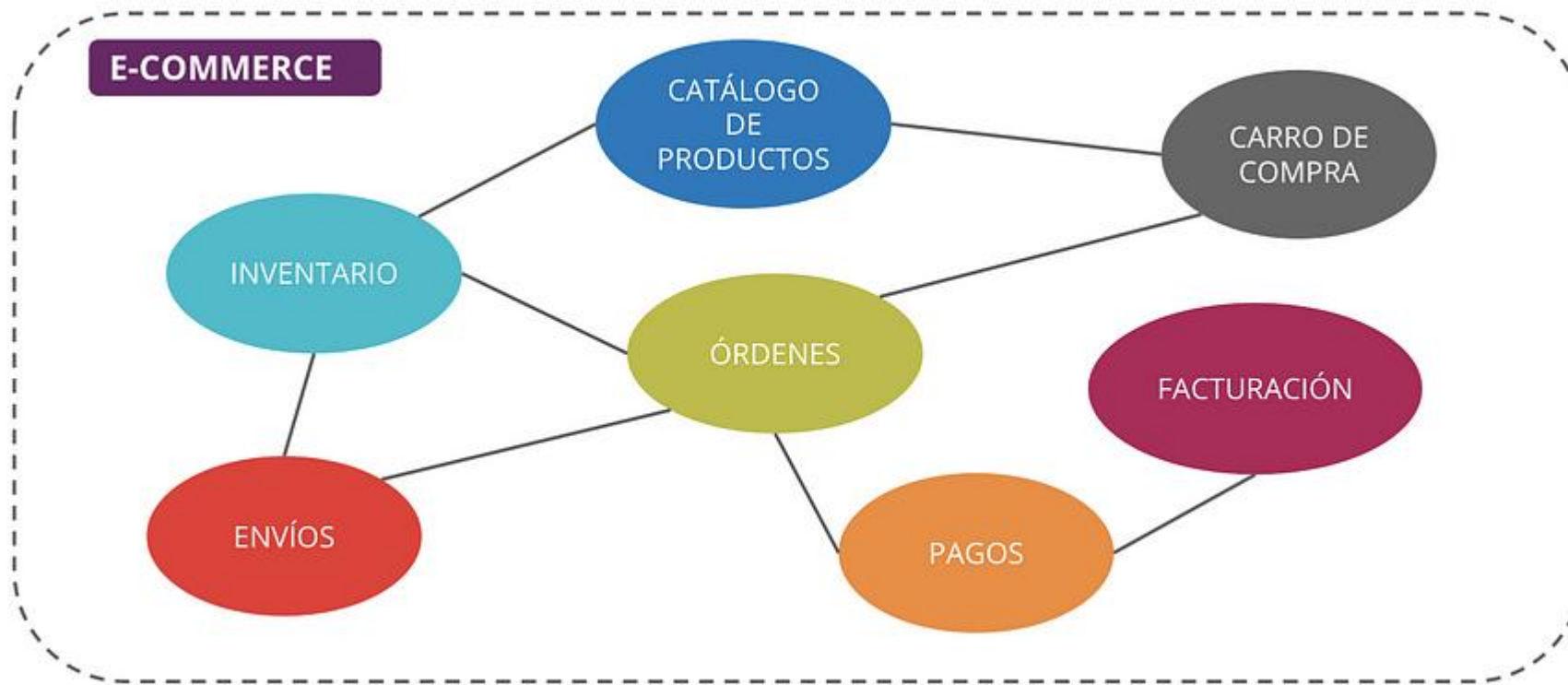
Es importante mantener un **lenguaje común entre los expertos de negocio y expertos de tecnología**, para lograr trasladar estos conceptos al código fuente, de forma tal de que nuestro producto de software refleje los conceptos que son hablados comúnmente por el negocio.



Ejemplos: Los acrónimos de negocio tienen que estar documentados en un lugar accesible, como confluence u otra wiki que haya sido creado en conjunto con las áreas involucradas.

• Bounded Context

Los **bounded context** son el patrón principal en Domain-Driven Design. Un Bounded Context es un espacio delimitado donde un elemento del negocio tiene un significado perfectamente definido.



Bounded context en un e-commerce..

- El contexto es definido por los términos usados en las charlas, documentación, reuniones, etc.

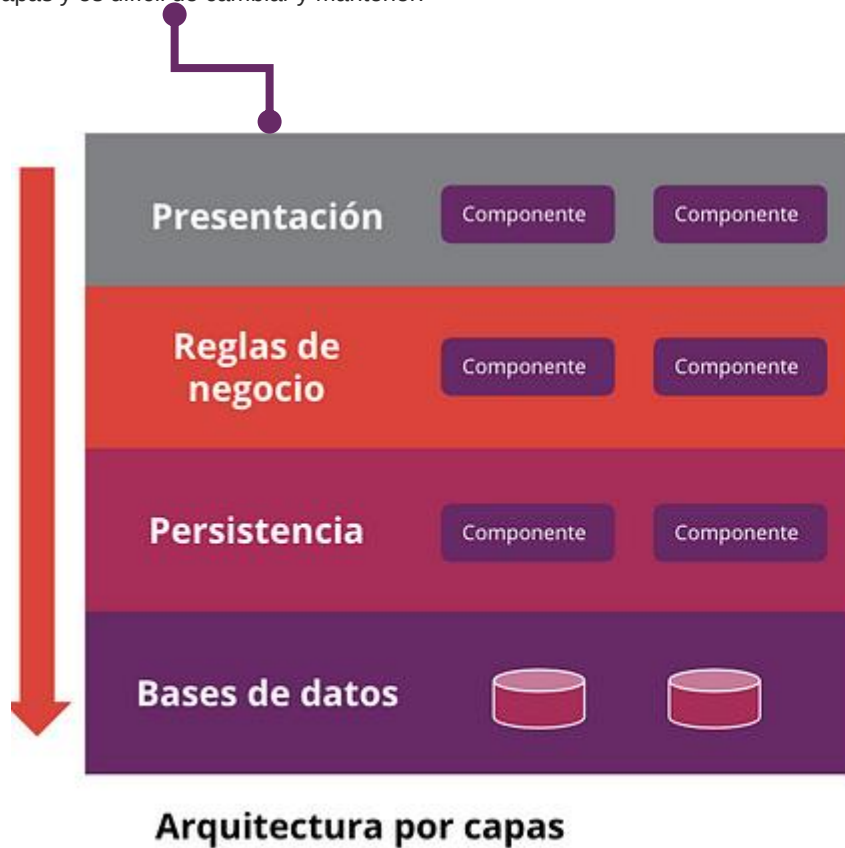
• Bounded Context

Un producto en el contexto de **envío** es diferente a un producto en el contexto de **inventario**. Aquí es donde toma más fuerza la idea de que el significado varía de acuerdo del contexto, pero también podemos observar que podría verse afectado el código (por una posible duplicación), pero podemos compararlo con la disminución de un alto acoplamiento. Además, tenemos la libertad de evolucionar de acuerdo al contexto.

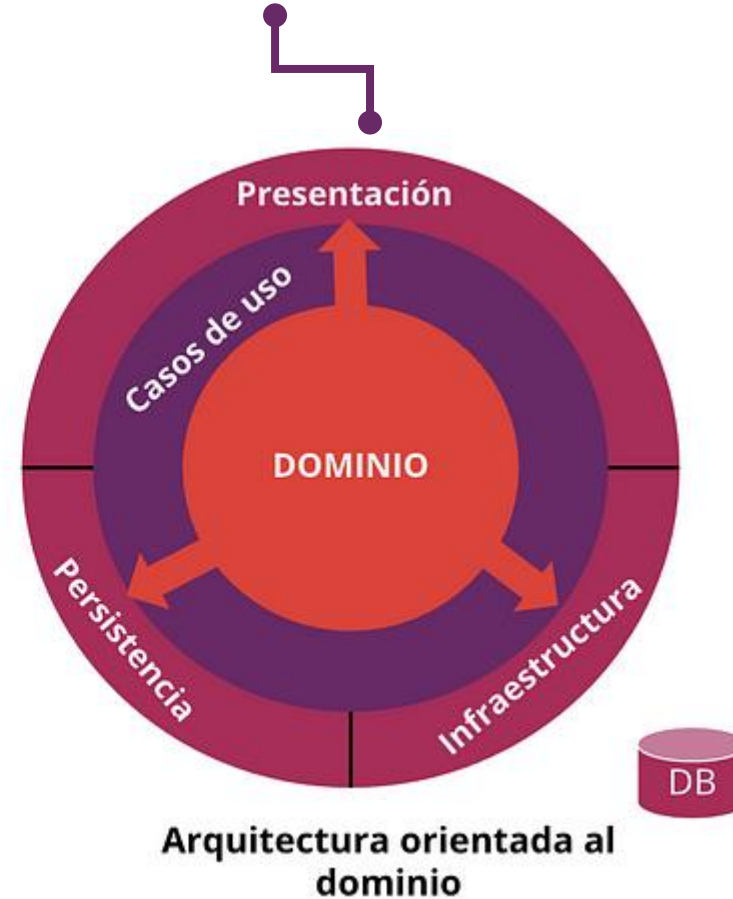


• Arquitectura Capas vs DD

En la arquitectura por capas, la dependencia va desde la capa de presentación hasta la capa de persistencia, lo cual está bien para aplicaciones cuyo grado de complejidad es bajo, pero a medida que el producto crece y su complejidad aumenta, las reglas de negocio quedan agrupadas a otras capas y es difícil de cambiar y mantener.



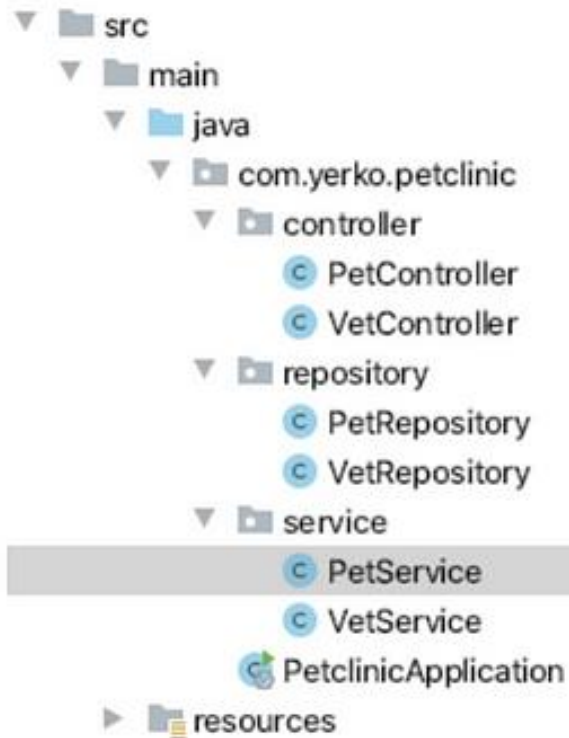
En la arquitectura centrada en el dominio, las dependencias se invierten y es el dominio el centro de todo, por lo cual las entidades, casos de uso o reglas de negocio no dependen de tecnología y tampoco de agentes externos, por lo cual es más fácil adaptarse al cambio y evolucionar.



Representación de distintas arquitecturas, izquierda por capas, derecha orientada al dominio.

• Arquitectura Capas vs DD

Arquitectura por capas



¿Qué hace la app?

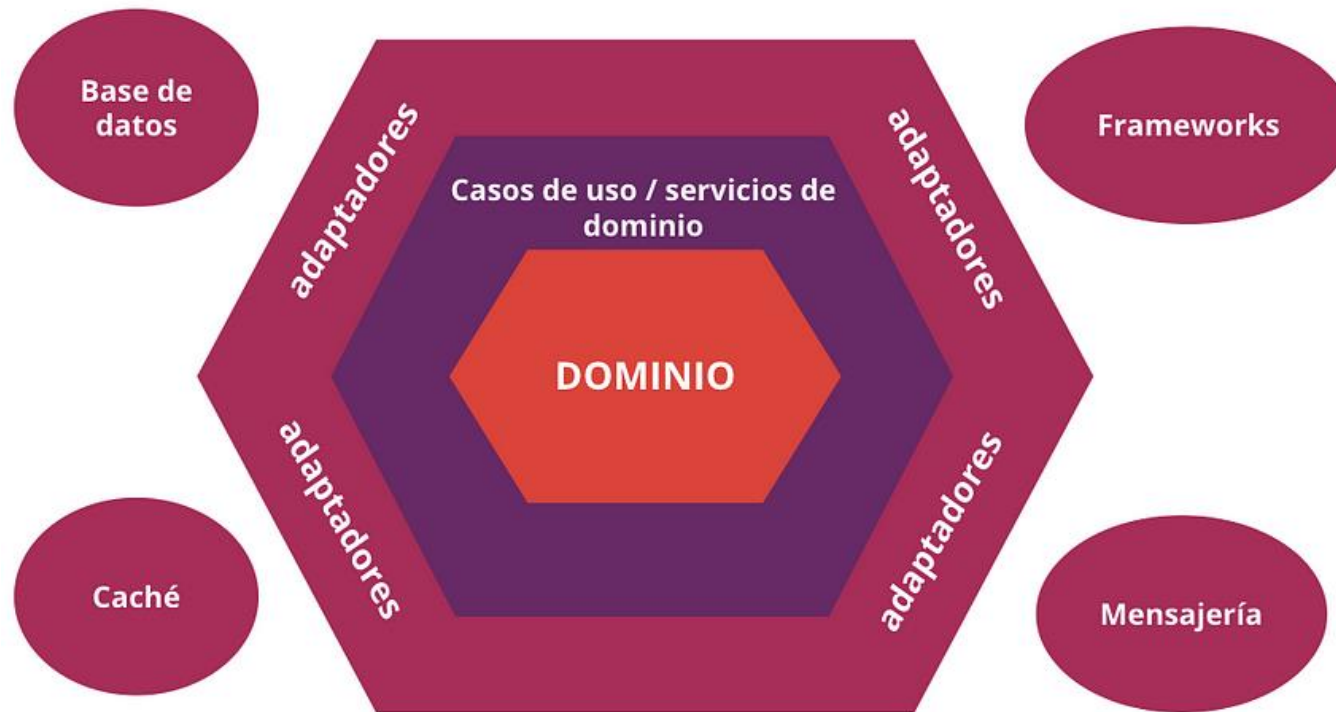
Centrada en el dominio



Representación de un caso real sobre las diferencias entre arquitectura por capas y centrada en el dominio.

• Arquitectura Hexagonal

La **arquitectura hexagonal** es conocida también como la arquitectura de puertos y adaptadores y tiene la misión de desacoplar las capas de la aplicación. Se describe como puerto, la definición de una interfaz pública y adaptador a la especialización de un puerto para un contexto en específico.

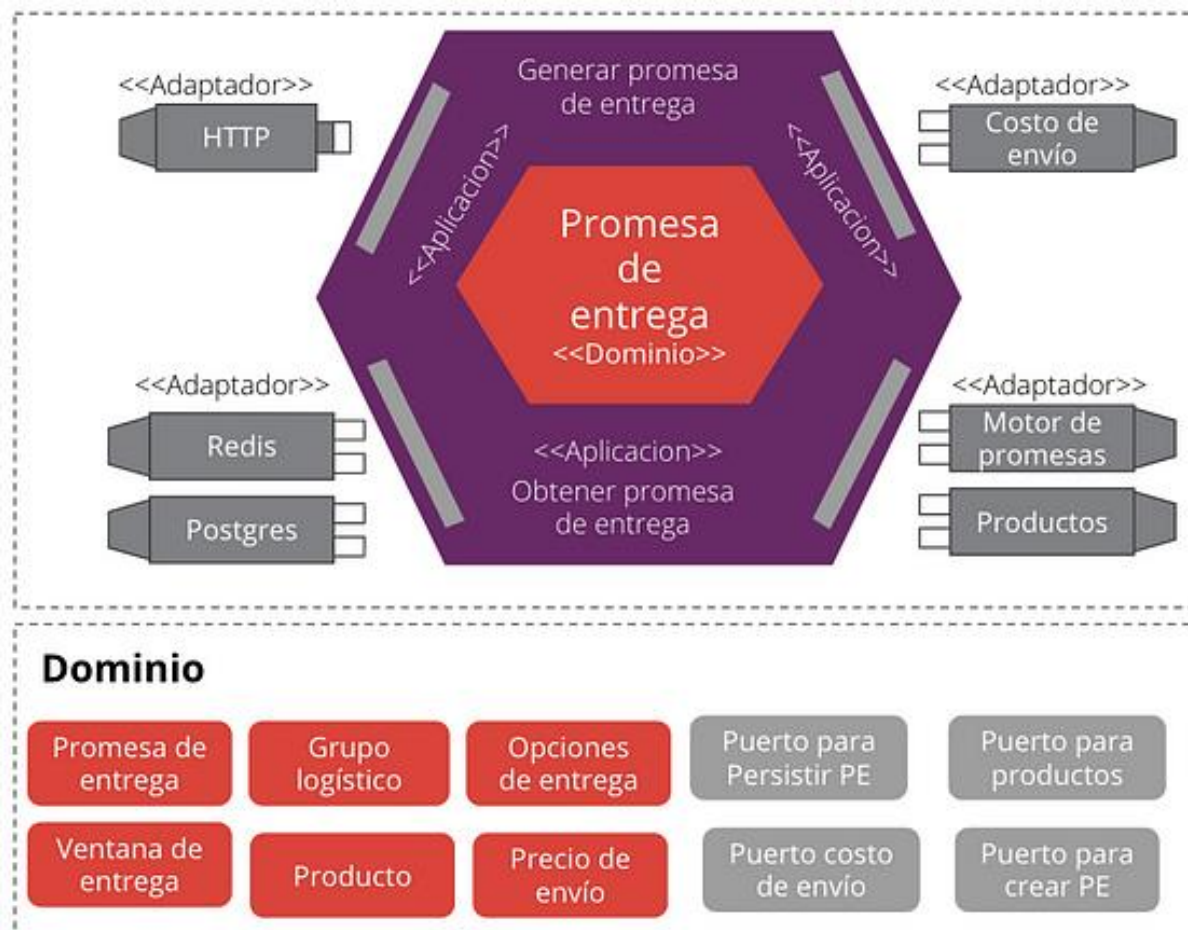
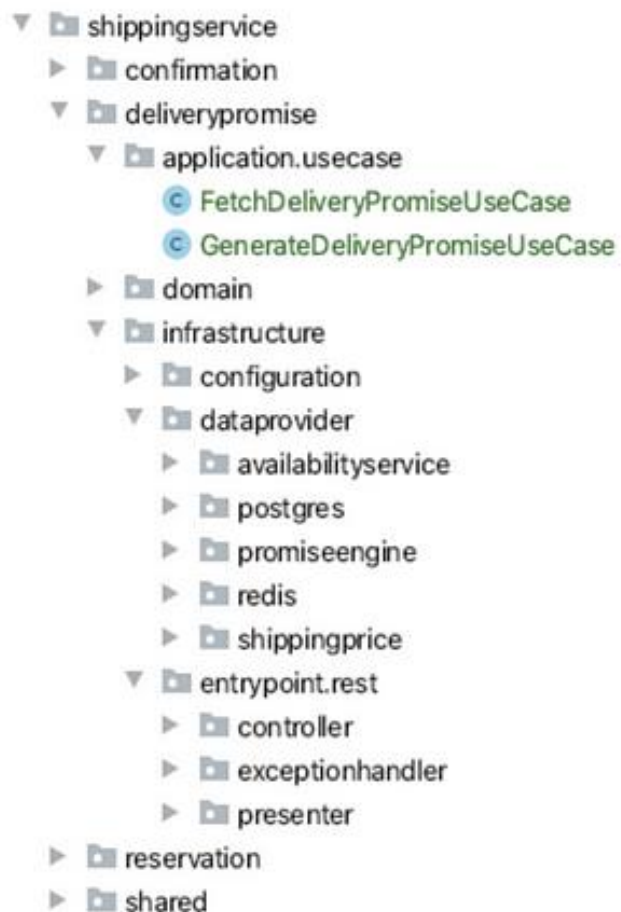


Ejemplo de una arquitectura hexagonal.

La elección de esta arquitectura se basa en sus características:

- Independiente del framework
- Testables
- Independientes de la UI
- Independientes de la base de datos
- Independiente de agentes externos
- Más tolerantes al cambio
- Reutilizable
- Mantenable

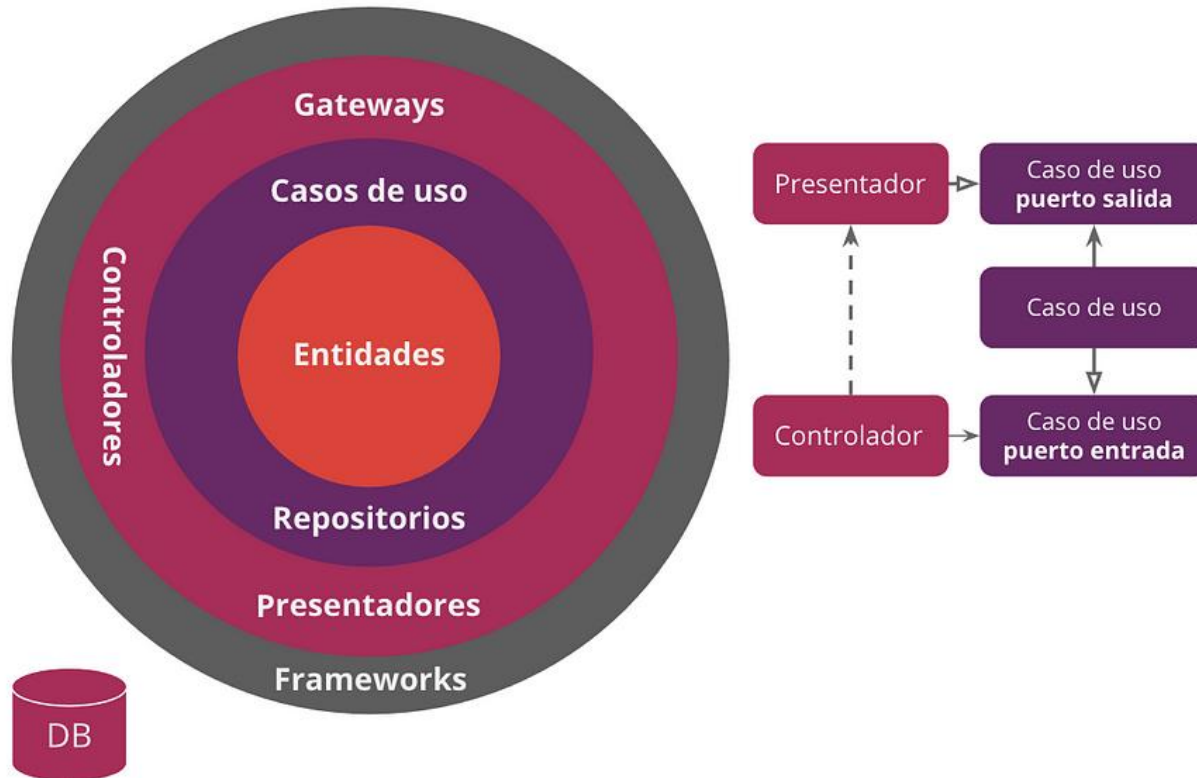
• Arquitectura Hexagonal



Descripción de una promesa de entrega aplicado a un e-commerce

• Arquitectura Limpia

La **arquitectura limpia** es una filosofía de diseño de software que separa los elementos de un diseño en niveles de anillo. La regla principal de la arquitectura limpia es que las dependencias de código sólo pueden provenir de los niveles externos hacia adentro. El código de las capas internas no puede tener conocimiento de las funciones de las capas externas.

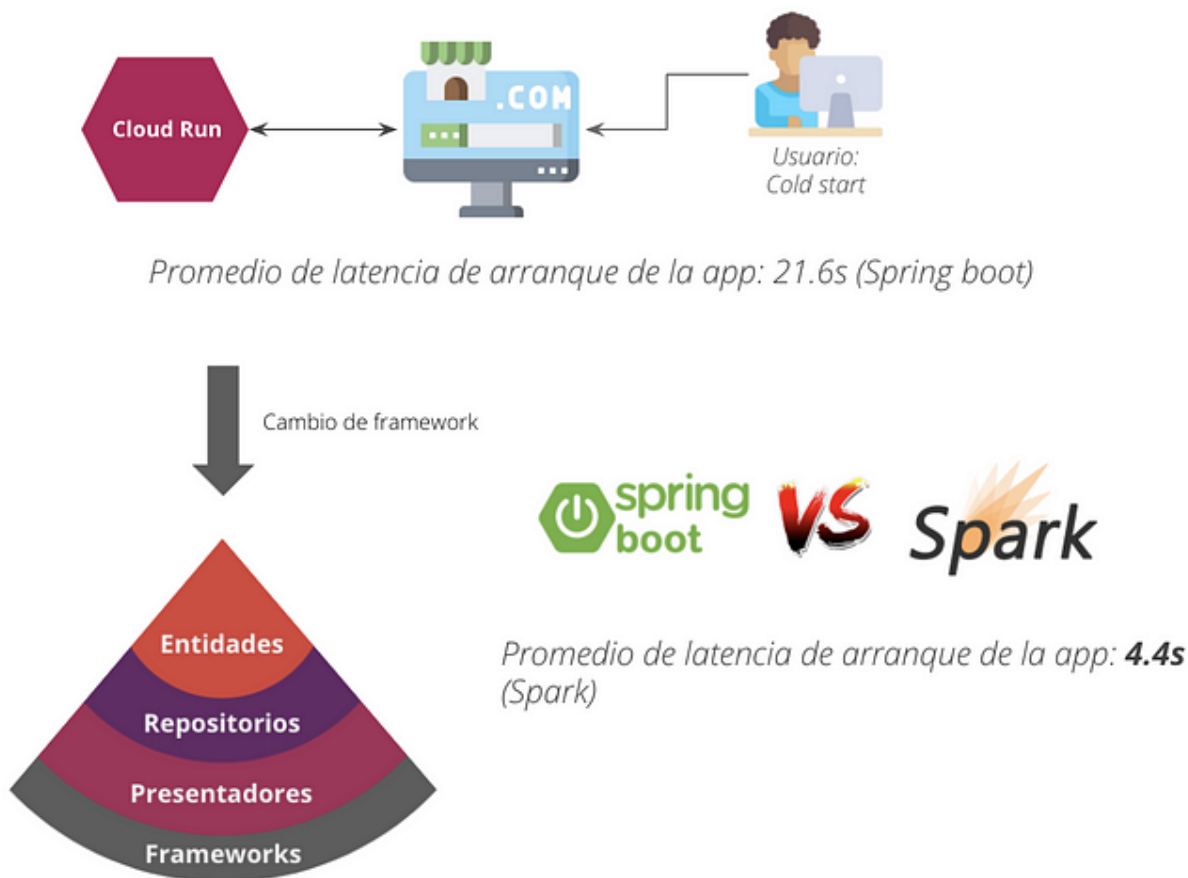


En este caso, la elección de este tipo de arquitectura se realiza debido a sus características:

- Independiente del Framework y la DB
- UI puede cambiar fácilmente
- Fácil testeo
- Reutilizable
- Mantenable

• Arquitectura Limpia

- ▶ authentication
- ▼ sales
 - ▶ configuration
 - ▶ dataprovider
 - ▶ infraestructura
 - ▶ orchestrator
 - ▶ usecase
- ▼ domain
 - ▶ entity
 - ▶ exception
 - ▶ provider
- ▼ infraestructura
 - ▶ dataprovider
 - ▼ web
 - ▶ configuration
 - ▶ controller
 - ▶ exception
 - ▶ model
- ▼ usecase
 - ▶ orchestrator
 - ▶ CreateSaleUseCase
 - ▶ GetSalesUseCase
- ▶ SalesApplication



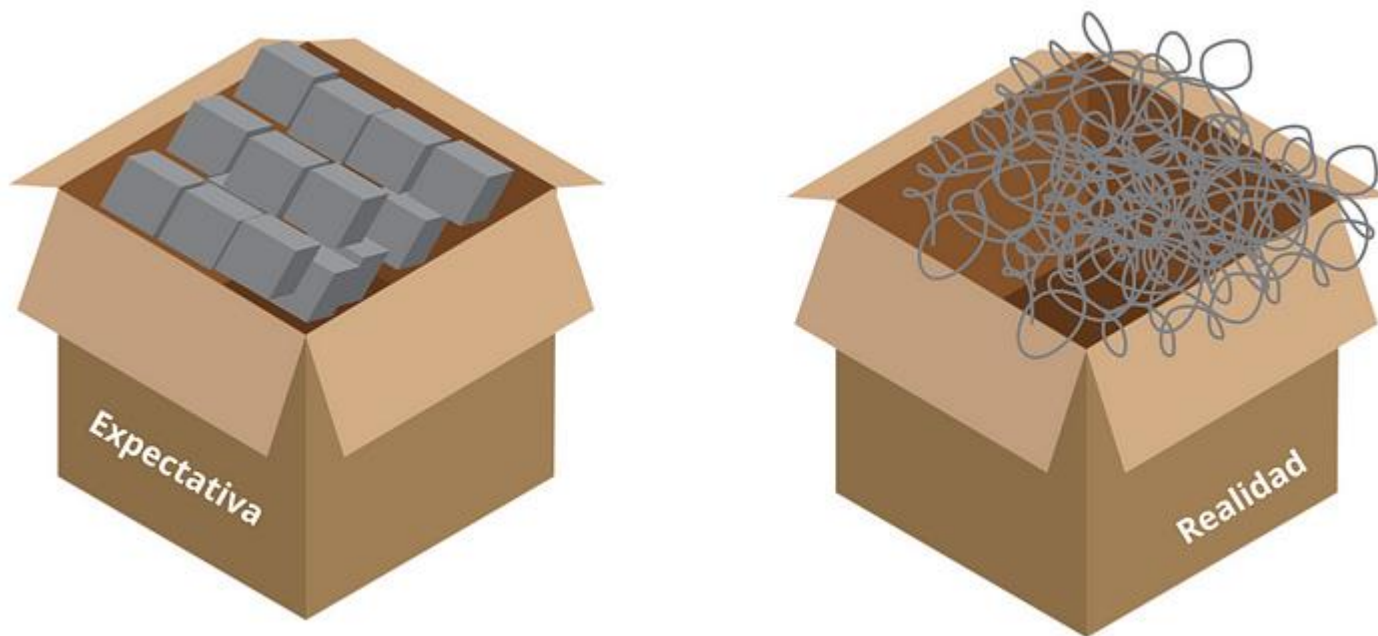
Caso: Alto tiempo de arranque al utilizar un framework robusto como Spring, sin necesitarlo.

Solución: Cambiar el framework.

Como funcionó esto: El desarrollo fue a nivel de framework con un par de líneas de código que logrando hacer un cambio “limpio” sin mayor impacto a las capas inferiores. Al no contar con spring security como librería dentro del framework, se necesitó implementar un middleware, además de generar un archivo de configuración de rutas, pues spring boot trae incorporada esta funcionalidad. Al hacer las pruebas de arranque en un estado de latencia, vimos que nuestro cambio disminuyó el tiempo de arranque de la aplicación de 21s a 4.4s

Representación del resultado de un cambio de framework gracias a la arquitectura limpia.

• ¿Cómo comenzar una DDA?

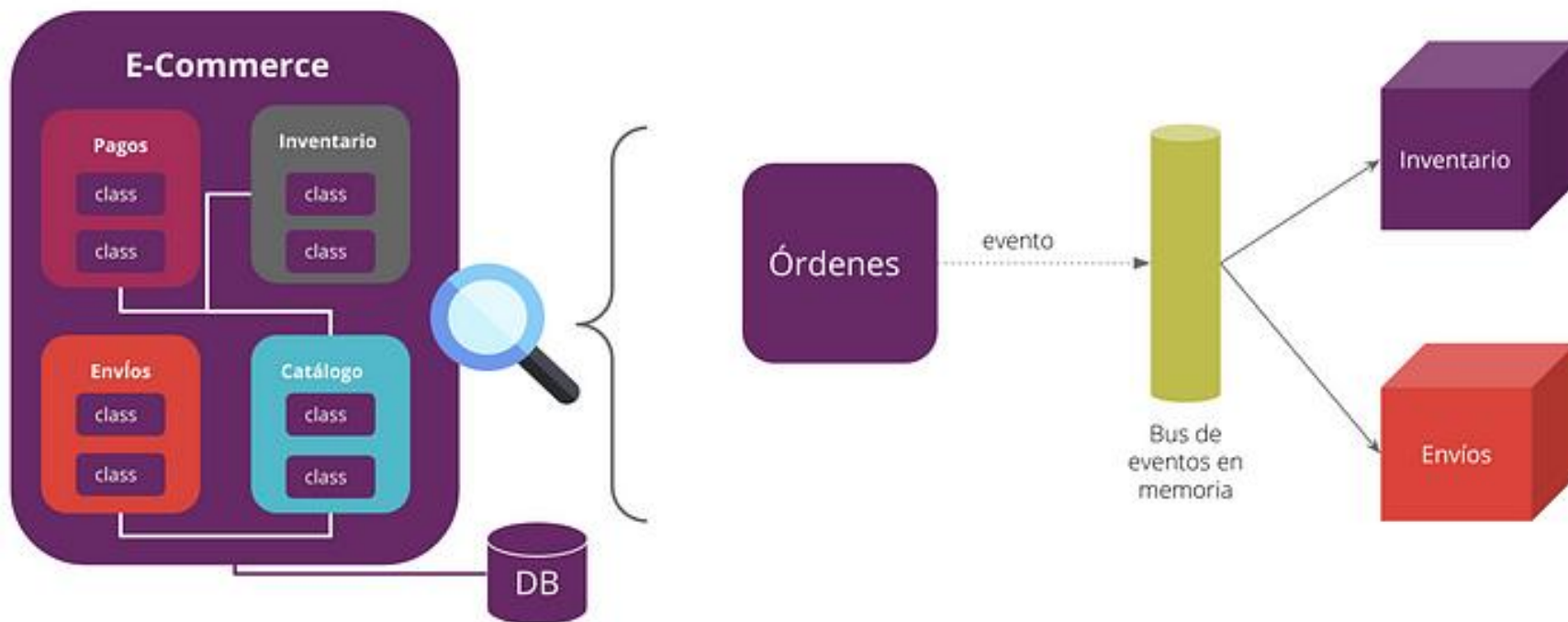


Representación de la expectativa de monolito modular a la realidad de una “big ball of mud”

La expectativa es tener un monolito ordenado en torno a las capacidades de negocio, pero la realidad es que tenemos dependencias por múltiples lugares mucho acoplamiento que es difícil de evolucionar o cambiar, ya que un cambio en un lugar puede afectar el comportamiento de otra funcionalidad, lo que dificulta el proceso de cambios y de extracción de capacidades de negocio del **monolito a microservicios**.

• ¿Cómo comenzar una DDA?

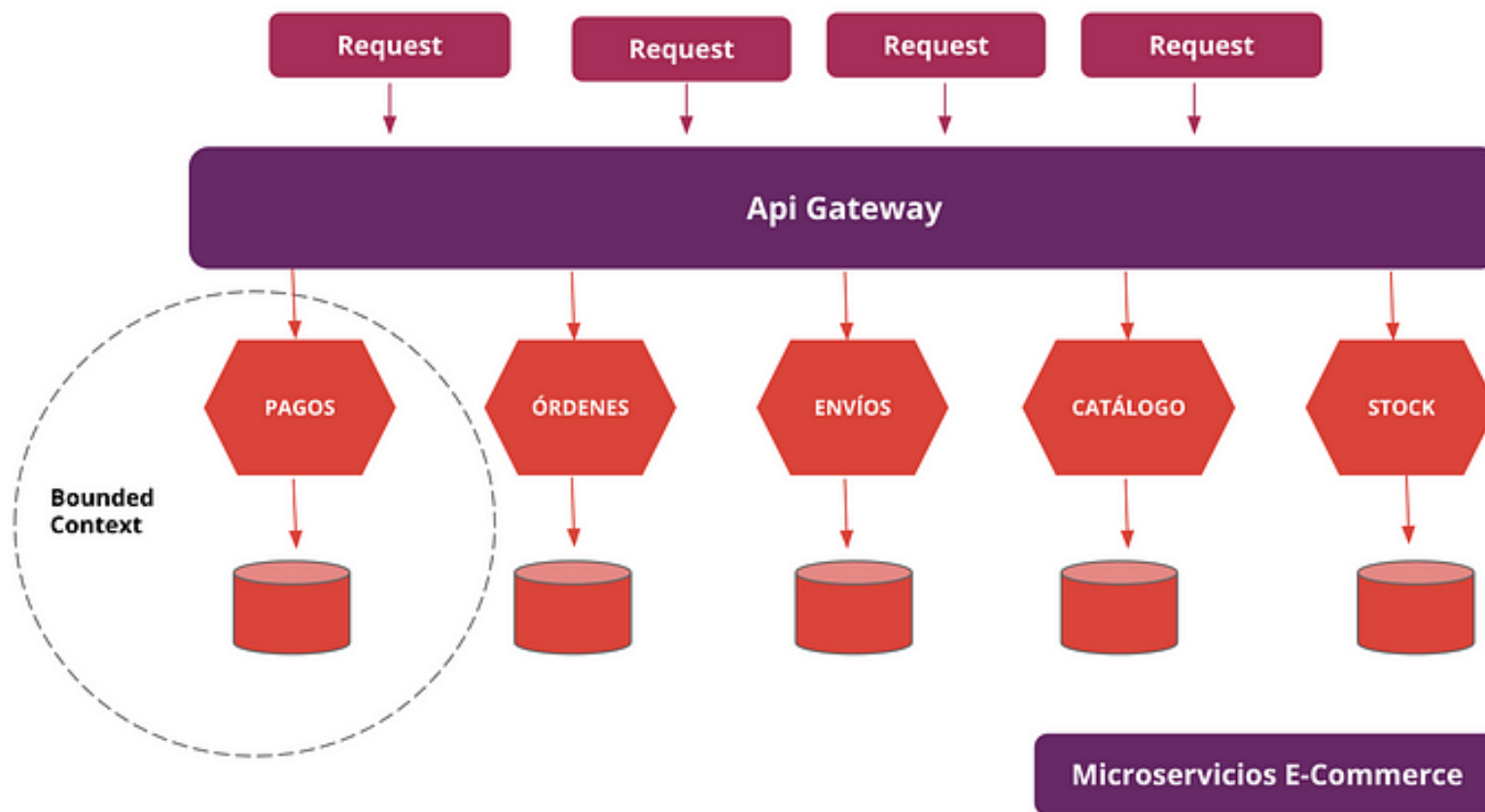
Una estrategia para lograr **evitar dependencias y acoplamiento** es el uso de monolitos modulares donde encapsulamos en módulos los distintos bounded context o dominios de nuestra aplicación, una estrategia para tener dominios aislados es lograr la comunicación entre módulos mediante bus de eventos en memoria, de esta forma logramos evitar acoplamiento entre módulos, y a futuro es más fácil transicionar a microservicios.



Representación de un monolito modular de un e-commerce

• ¿Cómo comenzar una DDA?

Utilizando **event storming y bounded context** podemos definir microservicios, aunque no siempre vamos a tener una relación de 1 a 1 con respecto a microservicios/bounded context, y es posible que en un bounded context existan múltiples sub-dominios que se pueden traducir en otros microservicios.



¿cuáles son los principios para trazar los límites del servicio?

• Principios para trazar límites de servicios



¿Cómo saber si
necesito un
monolito o
microservicios?

• Principios para trazar límites de servicios

Elige **monolito** cuando:

- No existe una cultura de DevOps, es importante que las personas sean capaces de desarrollar esta habilidad antes de migrar del monolito
- Estás prototipando una idea y necesitas llevarla a producción de forma rápida
- Código existente presente en una sola unidad de despliegue

Elige microservicios cuando :

- Tu equipo es maduro en el negocio y la empresa tiene una cultura de DevOps
- Las personas desarrolladoras de tu equipo tienen experiencia en microservicios y entienden los conceptos de DDD (a veces adoptamos lo que está de moda sin leer los fundamentos, lo que puede llevar a equivocaciones de base que pueden impactar el negocio)
- Cuando necesitas unidades de despliegue independientes y escalar de forma horizontal



03

- Modelando la Arquitectura de Software

• Modelando la arquitectura del software

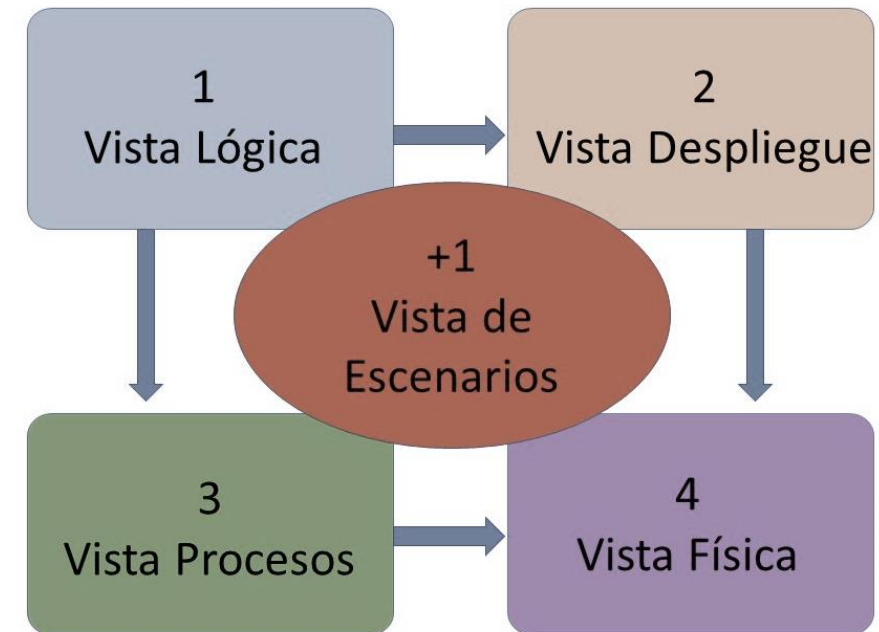
Al desarrollar software, es importante tener un plan claro y comprender cómo funcionarán juntos los diferentes componentes del sistema. La documentación y el diagrama nos ayudan a lograr esto al proporcionar una representación visual de la arquitectura del software. Una de las formas más conocidas de visualizar la arquitectura de software es el **modelo 4+1**.

Al momento de plantearse el trabajo o una tarea compleja relacionada con la **construcción de un sistema** los **diagramas o notaciones** nos permiten mostrar determinados aspectos interesantes del mismo con un determinado nivel de abstracción.

Los **diagramas ofrecen vistas parciales** y abstraen el sistema a desarrollar, por lo tanto, no contienen cada detalle del sistema en sí, sino que es una aproximación a la realidad que éste representa.

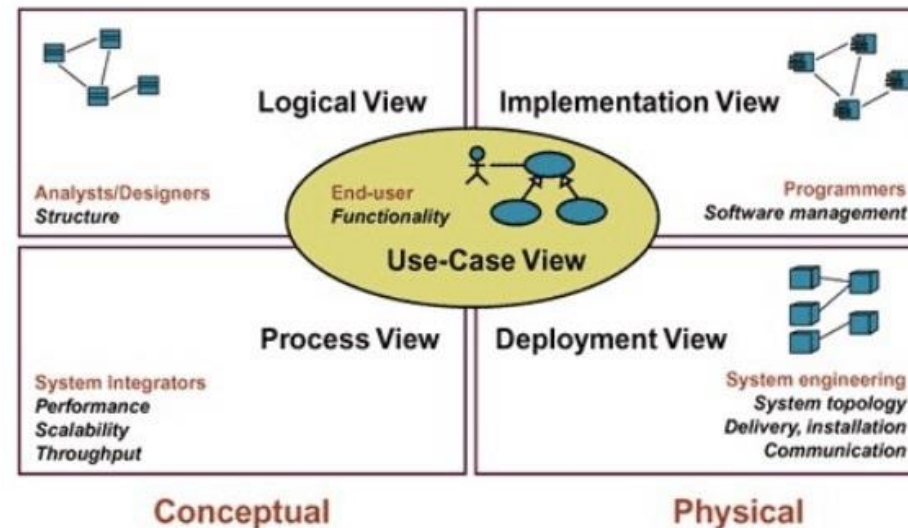
- **Modelo 4+1**

- El **modelo de vistas 4+1**, nos permitirá modelar nuestro software tomando como referencia un **enfoque centrado en la vista de los escenarios** como representación de las funcionalidades del software y a partir de esa definición, obtener visiones en diversos ámbitos del producto de software.

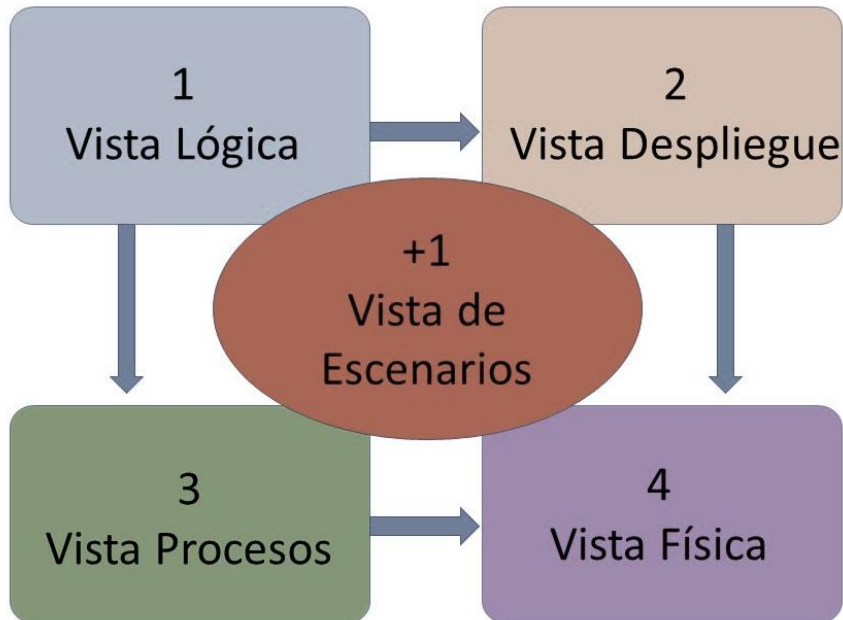


- **Modelo 4+1**

- El **modelo 4+1** es un modelo de vistas diseñado por el profesor **Philippe Kruchten** y se utiliza para **describir la arquitectura de un sistema** de software basado en el uso de múltiples puntos de vista, una vista nos indica **que es lo que se debe modelar** y un **punto de vista** son las **notaciones o normas para documentar** una o más vistas.



- **Vistas del modelo 4+1**



- Las vistas consideran **4 ámbitos: vista lógica, vista de procesos, vista de despliegue y vista física**, finalmente se adiciona (más uno) una vista que tiene la **función de relacionar las 4 vistas antes citadas** y que se denomina **vista de escenarios**

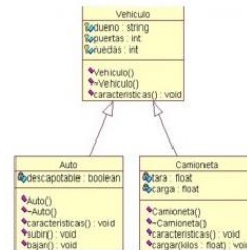
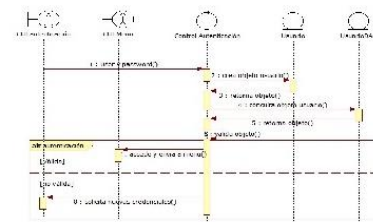
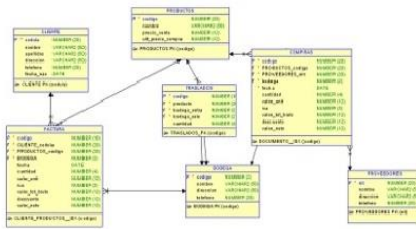
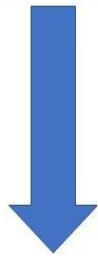
- **Vista Lógica**



- La **vista lógica** representa la estructura y función que el sistema proporcionará a los usuarios finales, es decir, se ha de representar **lo que el sistema realizará y las funciones y servicios que ofrece.**



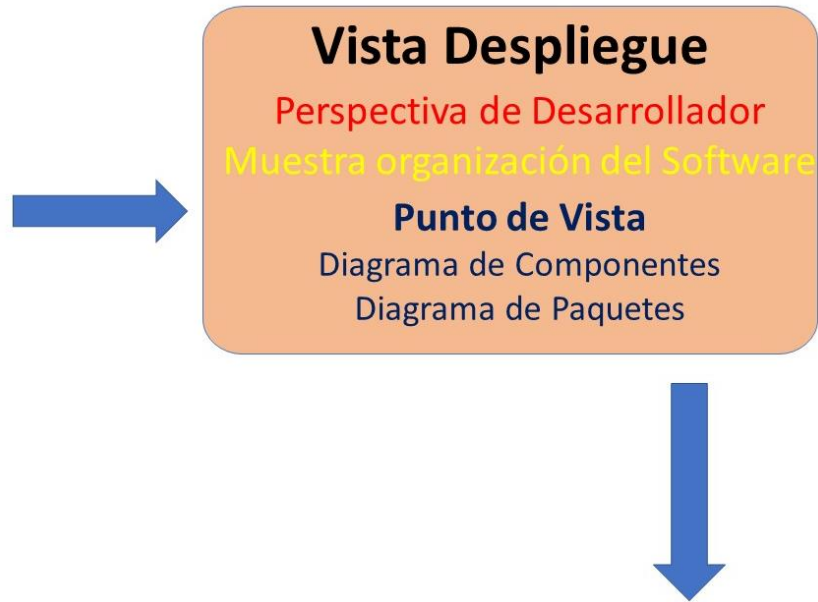
• Vista Lógica



Para completar la documentación de esta vista se pueden utilizar como punto de vista la notación que nos proveen los **diagramas de clases, de comunicación y de secuencia de UML**, también se pueden considerar como otro punto de vista la notación que proveen los **modelos de datos entidad-relación**.

La vista lógica está orientada a proveer información a los usuarios y todos quienes requieran conocer las funcionalidades del software.

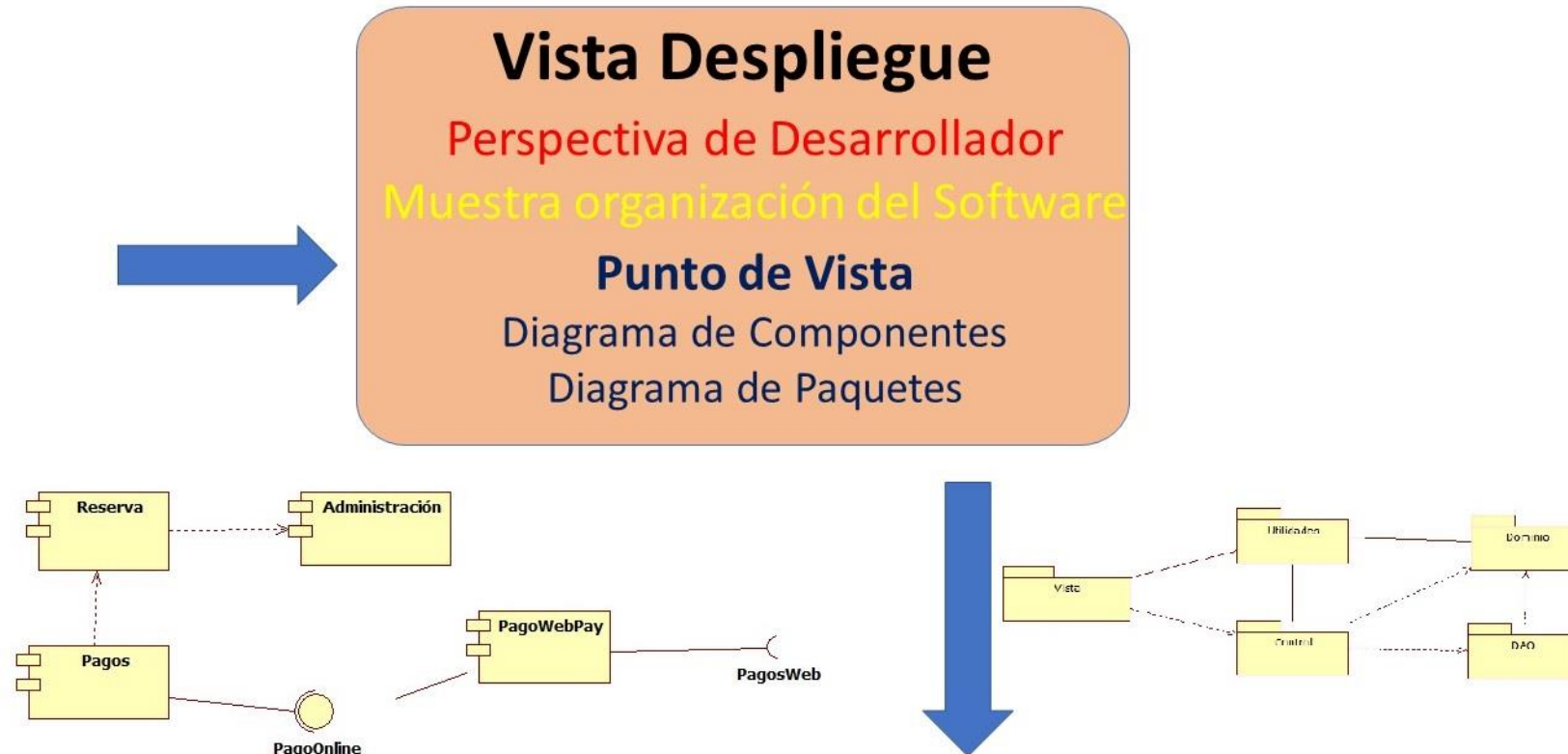
• Vista Despliegue



- La **vista de despliegue** muestra cómo está **dividido el sistema de software**, tanto sus **componentes como las dependencias** que hay entre estos componentes, además muestra como está **organizado el código fuente** y bibliotecas de funcionalidades adicionales o incluidas en la aplicación, mediante UML se puede utilizar **diagramas de paquetes y diagramas de componentes**.

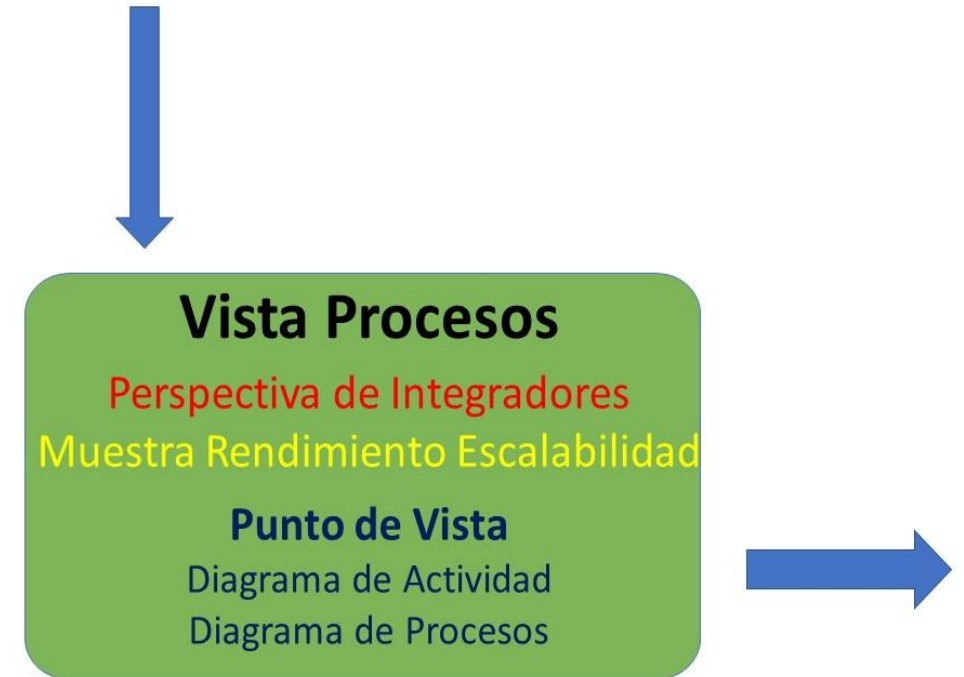
- **Vista Despliegue**

- La vista de despliegue muestra el software desde la **perspectiva** de un **desarrollador de software**.

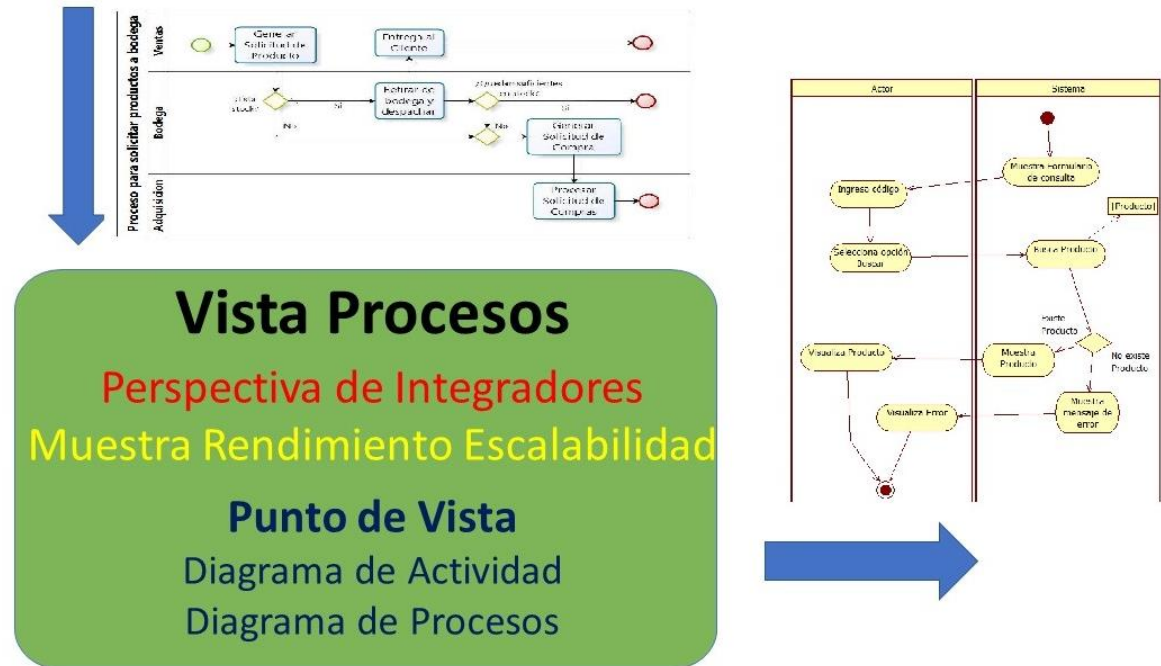


- **Vista de procesos**

- La **vista de procesos** muestra los procesos que hay en el sistema y la **forma en la que se comunican estos procesos**, los flujos de trabajo de componentes negocio y operacionales que conforman el sistema logrando establecer un **rendimiento y escalabilidad**.

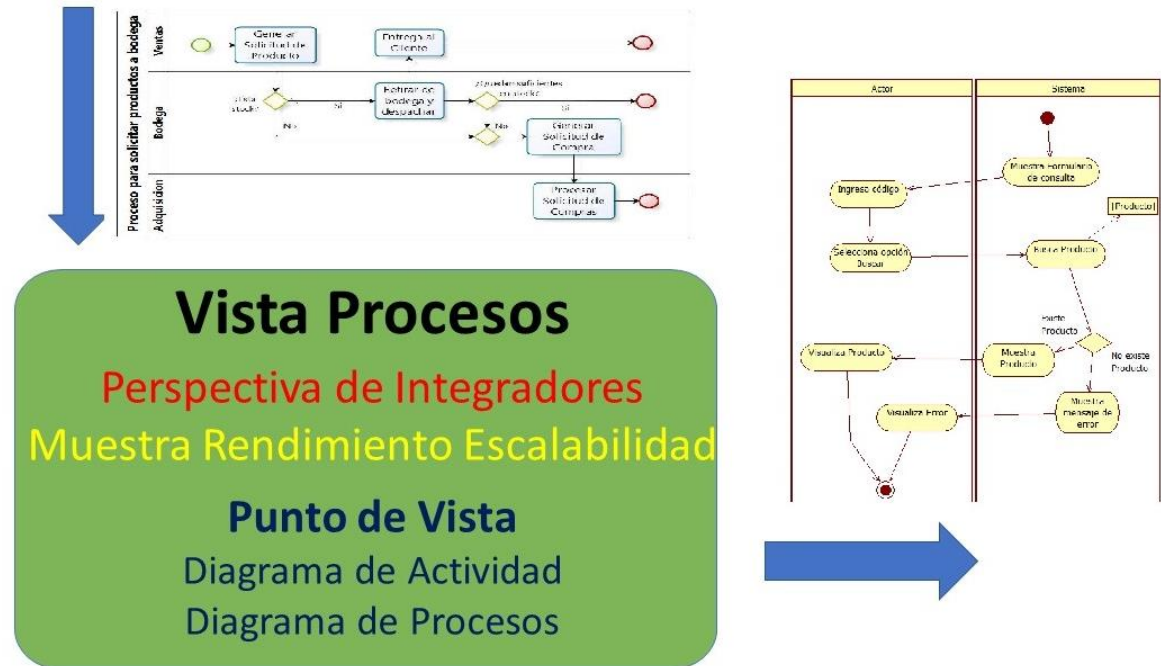


- **Vista de procesos**



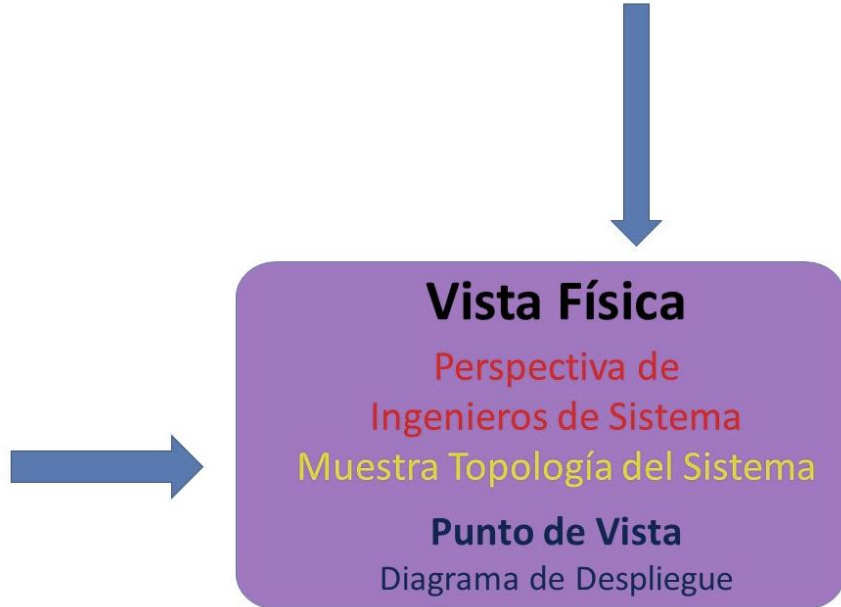
Se pueden utilizar diversos puntos de vistas para documentar la vista de procesos, por ejemplo, utilizando **notación de procesos** a través de diagramas **BPMN** y mediante UML, el **diagrama de actividades** es el punto de vista que más se ajusta para su representación.

- **Vista de procesos**



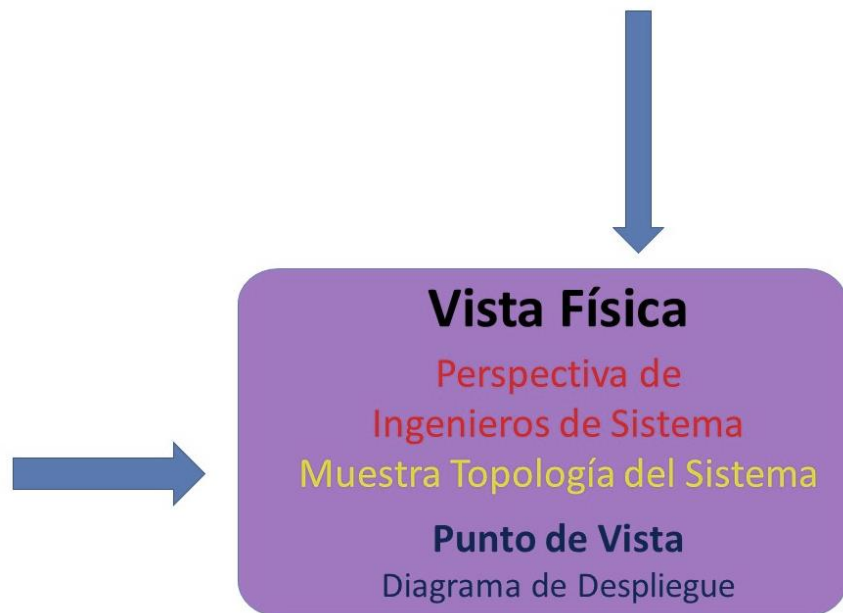
Esta vista es una **perspectiva** orientada a los **integradores** de la aplicación, que tendrán de sincronizar los diversos procesos del sistema para lograr su operación.

- **Vista física**



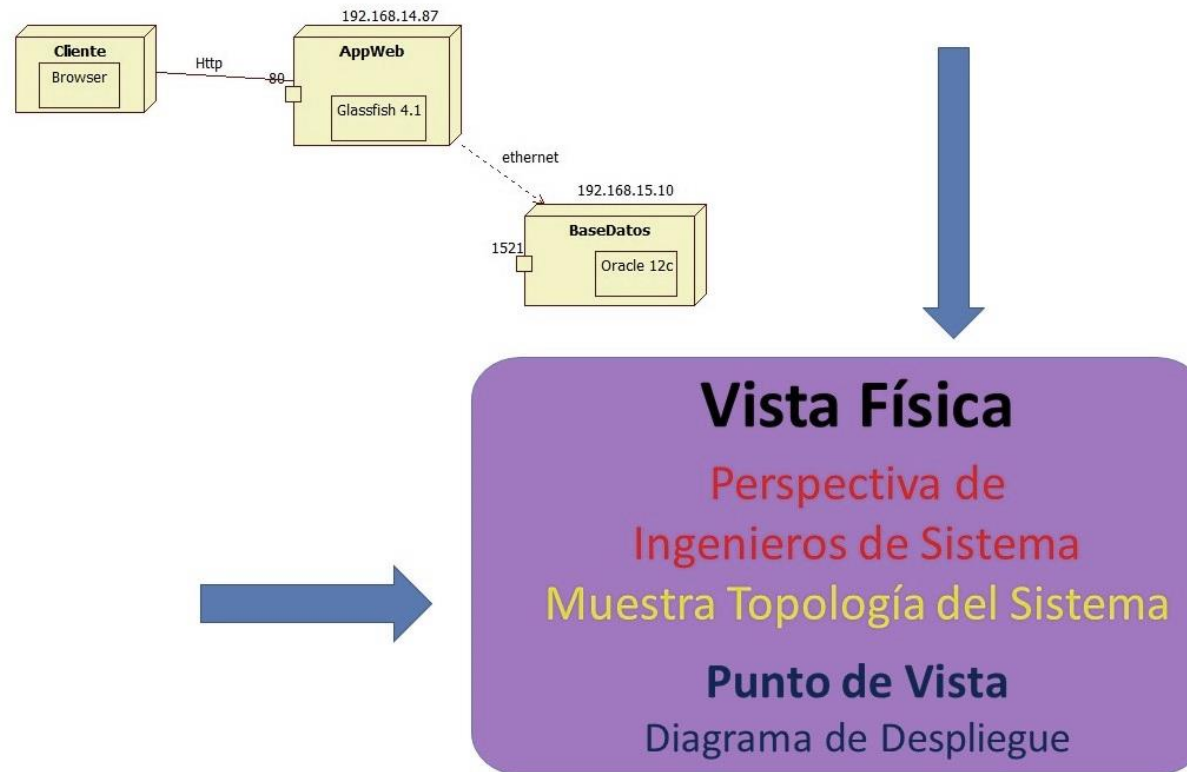
La **vista física** muestra los **componentes físicos del sistema** y las **conexiones entre dichos componentes**, los cuales conforman la solución de software, se incluyen además diversos **servicios** que la aplicación puede consumir.

- **Vista física**



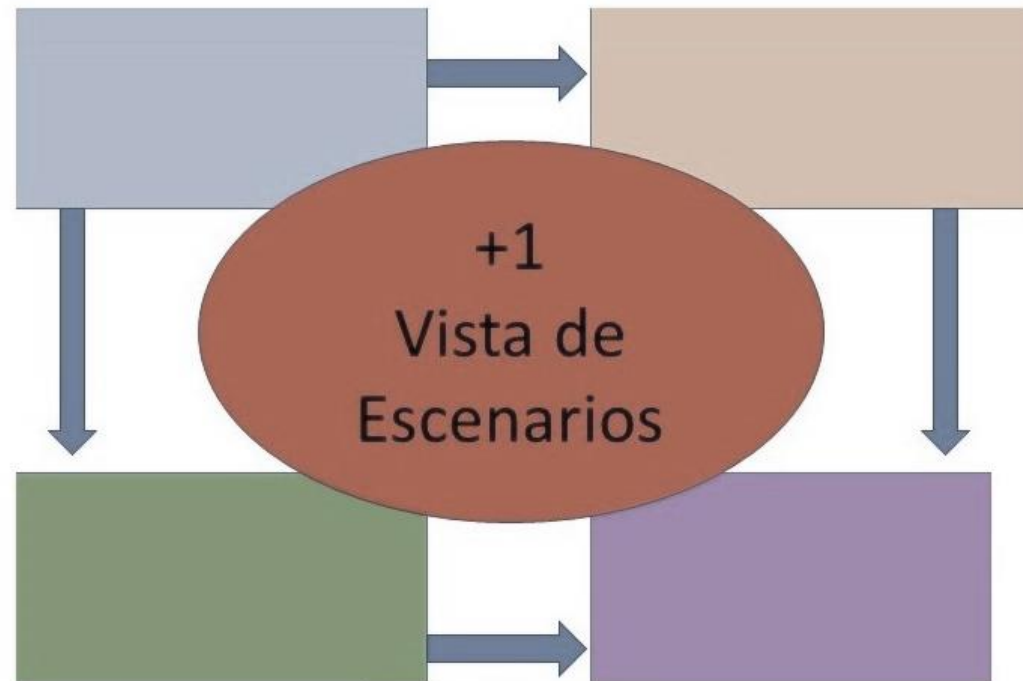
Esta vista está **orientada** a los **ingenieros de sistemas y de infraestructura**, los cuales deben integrar elementos de hardware, software y orquestar servicios para lograr soportar la ejecución de la aplicación y el almacenamiento de sus respectivos datos.

- **Vista física**
- El punto de vista puede estar representado por el **diagrama de despliegue de UML**.



- **Vista de escenarios**

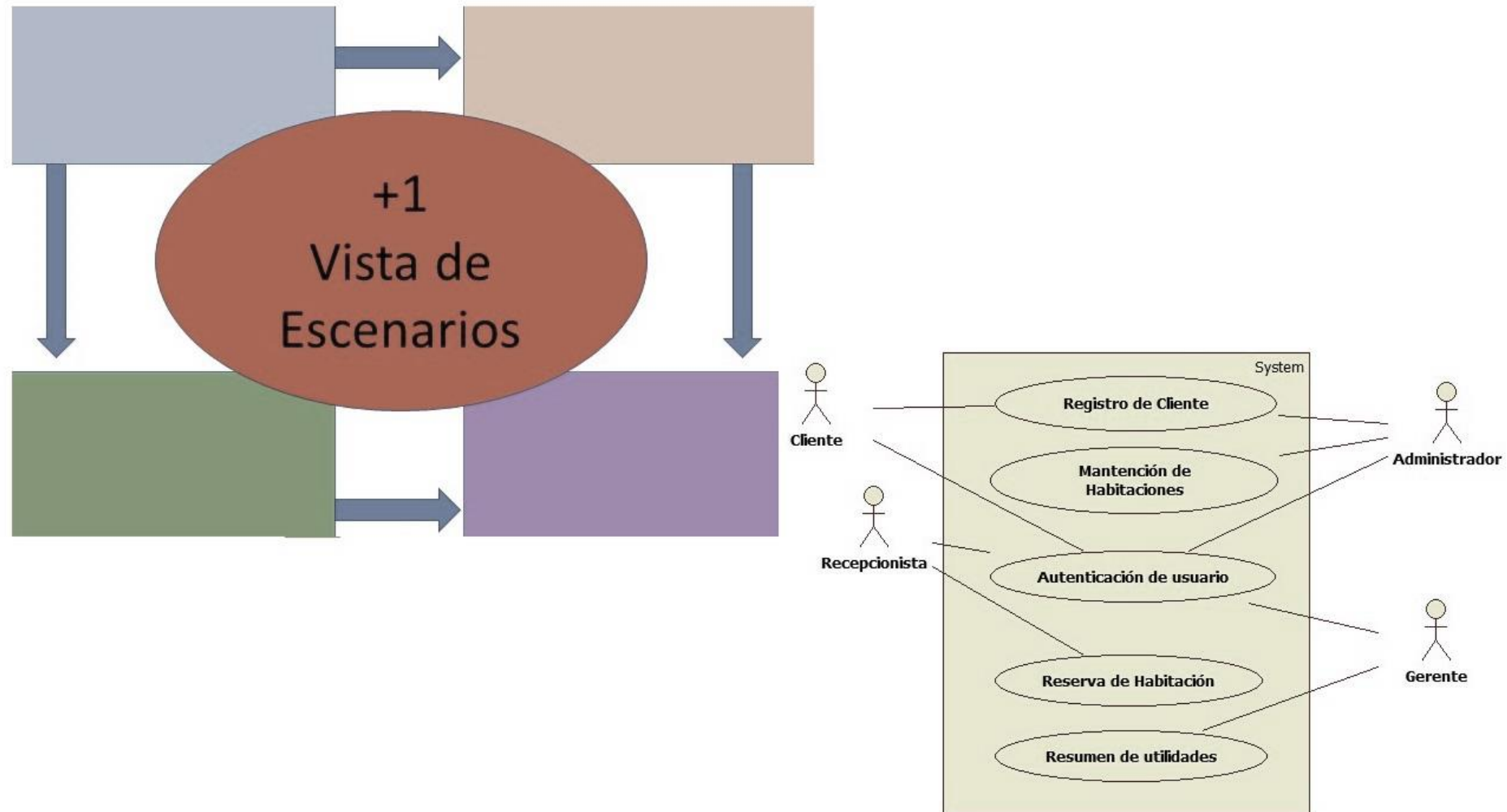
- La **última vista del modelo 4+1** se refiere a la **vista más uno**, que corresponde a la **vista de escenarios**, esta considera los aspectos relacionados a los **casos de uso** que serán cumplidos por el software y se **vincula con las otras 4 vistas**.



- **Vista de escenarios**

- Esto quiere decir, que a partir de los casos de uso surgen las **necesidades que debe satisfacer el software** y para lo cual el sistema debe lograr trabajar en forma integrada en diversos aspectos del software y de esa forma lograr sus objetivos como producto.
- Como punto de vista, se puede utilizar la **notación definida en el diagrama de casos de usos de UML**.

- **Vista de escenarios**



04

- Conclusiones y Resumen

• Resumen

- **Arquitectura de Software y Patrones:** La arquitectura de software define la estructura de un sistema, sus componentes y sus relaciones. Los patrones de diseño son soluciones recurrentes a problemas comunes en el diseño de software, proporcionando una guía reutilizable y probada para crear software robusto y mantenible.
- **Arquitectura Basada en Dominios:** Esta arquitectura organiza el sistema en módulos basados en dominios específicos del negocio, promoviendo la separación de preocupaciones y facilitando la alineación del software con las necesidades del negocio. Utiliza conceptos como el modelo de dominio y el lenguaje ubicuo para mejorar la comunicación entre desarrolladores y expertos del dominio.
- **Modelado 4+1:** El modelo 4+1 describe la arquitectura de software utilizando cinco vistas diferentes: lógica, de desarrollo, de proceso, física y de escenarios. Estas vistas permiten a los diferentes interesados (desarrolladores, usuarios, operadores) entender y validar el diseño del sistema desde múltiples perspectivas, asegurando una cobertura integral de los requisitos y funcionalidades.

• Bibliografía

- Fowler, M. (s/f). *Software Architecture Guide*. martinowler.com. Recuperado el 22 de junio de 2024, de <https://www.martinfowler.com/architecture/>
- Laso, J. (2021, agosto 4). *¿Qué es la arquitectura centrada en el dominio? - Javiera Laso*. Medium. <https://jlasoc.medium.com/qu%C3%A9-es-la-arquitectura-centrada-en-el-dominio-10542ea87afe>
- Kruchten, Philippe (1995, November) (Versión en español). [Planos Arquitectónicos: El Modelo de “4+1” Vistas de la Arquitectura del Software.](#)

DuocUC[®]

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS
ACREDITADO



DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN
INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.